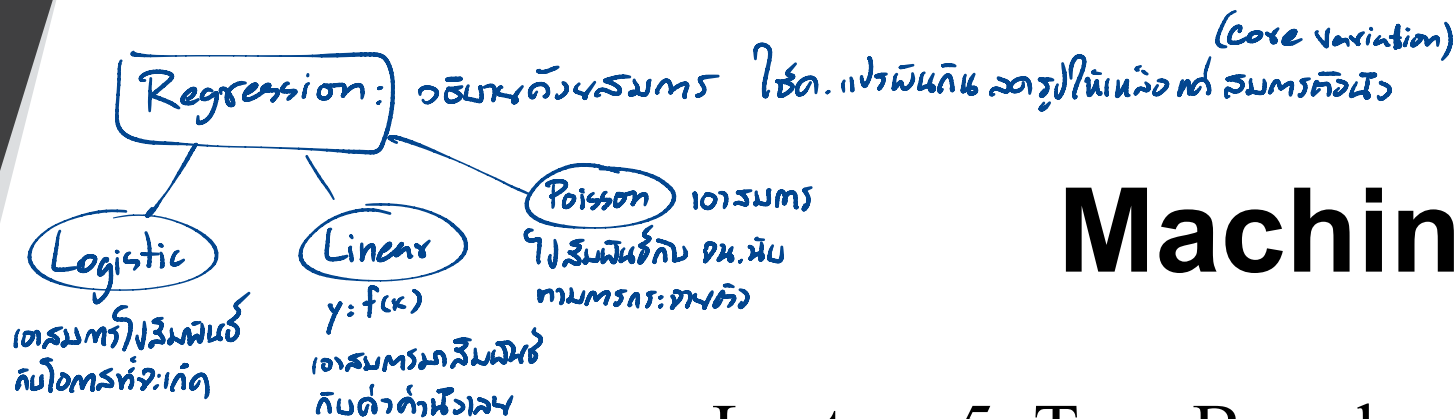




Machine Learning

020-800-1420

Lecture 5: Tree-Based and Ensemble Models

Ex: Random forest, Gradient Boosting, XGBoosting, Light GBM

1161 Lecture นี้เราจะไม่มาสอนวันนี้ !

เราจะไม่มาสอน !

Asst. Prof. Dr. Santitham Prom-on

M2 Exam Lecture 5-9 Tree base Basic clustering

- ข้อ 40 ข้อ

- Short Answer 10 ข้อ

อย่าลืมอ่านบทเรียน เรื่องการวิเคราะห์ข้อมูลด้วย

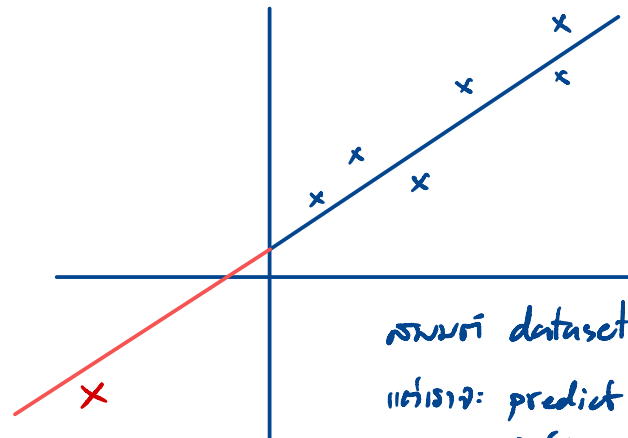
ดูเพิ่ม XG Boost และ Light GBM

Department of Computer Engineering, Faculty of Engineering
King Mongkut's University of Technology Thonburi

Lecture 5 - Topics

- Decision tree classification and regression
- Bagging
- Boosting

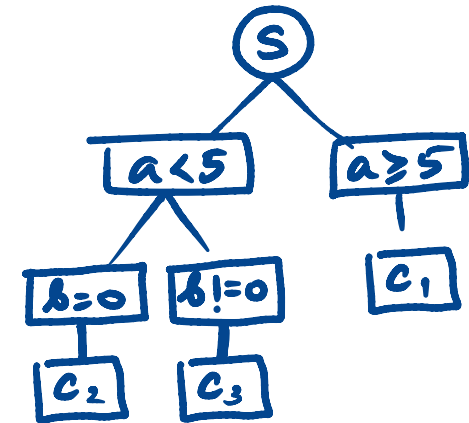
Extrapolation



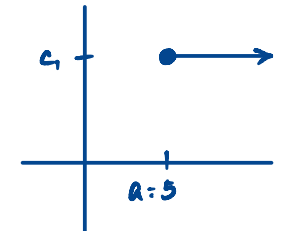
Regression
 ကိုယ် predictive
 model ကို
 predict ကို
 သိသော dataset
 ကိုယ် တွေ့ကြုံရင်

သိသော dataset ဝယ် Quadrant 1
 ကိုယ်တော့ predict a ကို Quadrant 3
 ကို predict လို $\therefore$ ကိုယ် သိသော model

Interpolation

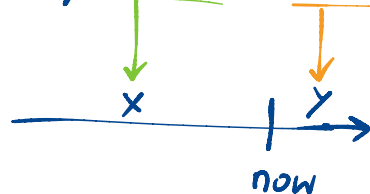


ကိုယ် a ကို
 5, 10, 15, 1000
 ကိုယ်တော့ c_1, c_2, c_3



predictive model:

ตัวแปรในข้อใด จะมาทำนายค่าของตัวแปร

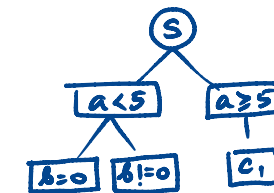


และลดทอนขนาดของข้อมูลได้

นี่ Lecture นี้จะสอนเกี่ยวกับ data

Decision tree (Non-linear Regression)

- Pros - Explainability : อธิบายได้ track ช้อนได้
 สามารถรู้ได้ว่า Rootnode มาถึง Output leafnode
 - Robust : Generalized ได้ดี เมื่อเทียบกับ model อื่น



Decision tree classification and regression

<https://drive.google.com/file/d/12eegDnFPtqL42rJcuUOVijbnFuet2uO/view?usp=sharing>

Predictive Modeling as Supervised Segmentation

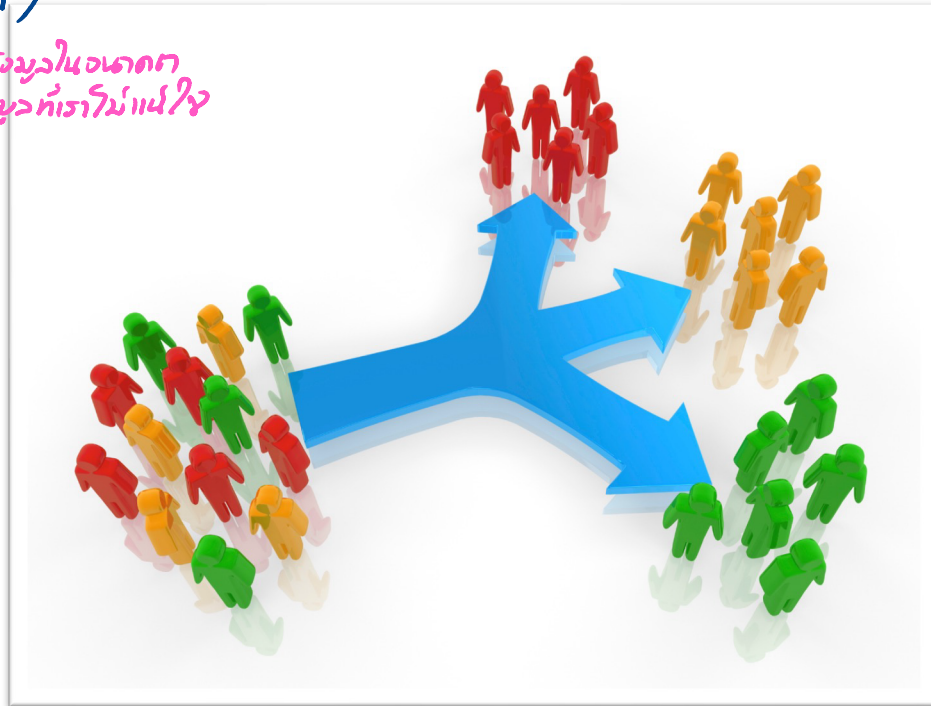
- How can we segment the population into groups that differ from each other with respect to some quantity of interest?

เราต้องการแบ่งประชากรออกเป็นกลุ่ม (เพศ, อายุ, สถานะสมรส, ฯลฯ)
 เพื่อหาความสัมพันธ์กับปริมาณที่สนใจ (Quantity of Interest)
 เช่น การหารายได้ในกลุ่ม แต่ละอาชีพ → ปริมาณที่สนใจคือ รายได้ของแต่ละอาชีพ

Quantity of interest

=

Things we would
like to predict or
estimate



Target and Attribute

Data record →

○ ၁ မှတ်တမ်း ၁

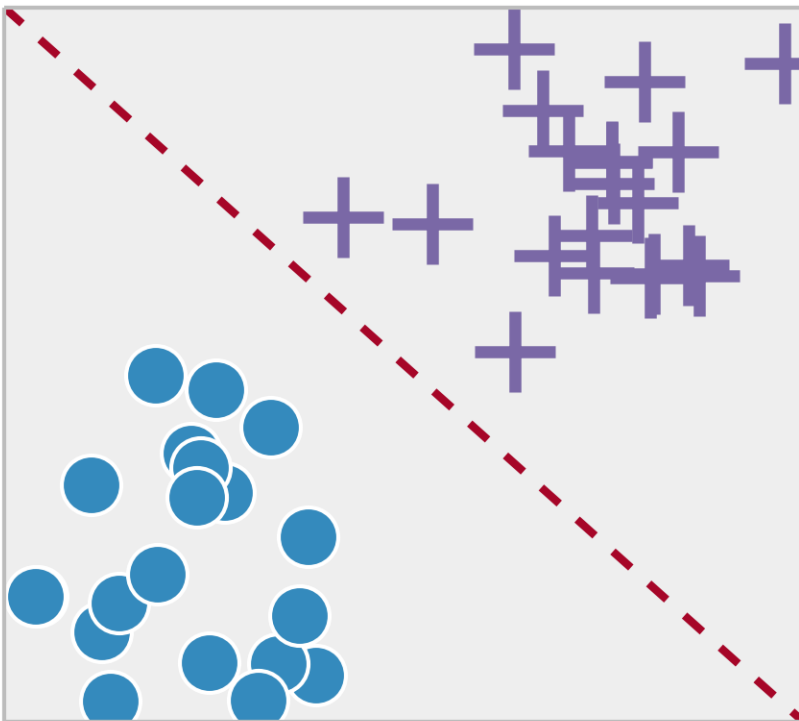
<i>target</i>	<i>Attribute 1</i>	<i>Attribute 2</i>	<i>...</i>	<i>Attribute k</i>
y_1	x_{11}	x_{12}	$\dots$	x_{1k}
y_2	x_{21}	x_{22}	$\dots$	x_{2k}
$\vdots$	$\vdots$	$\vdots$		$\vdots$
y_n	x_{n1}	x_{n2}	$\dots$	x_{nk}

Target attribute / class to predict

Attributes

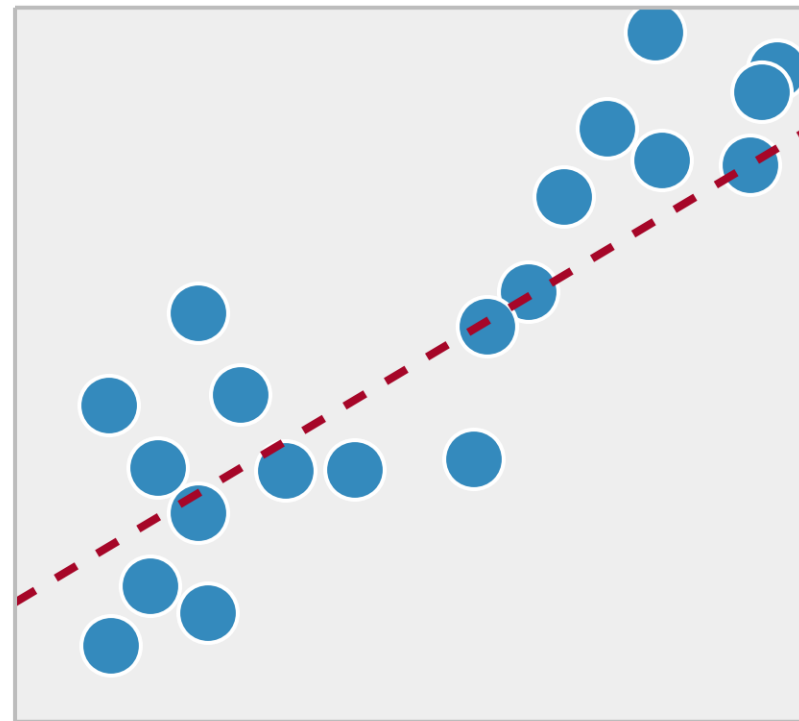
Classification vs Regression

Classification



Discrete Target (class)

Regression



Continuous Target (value)

Supervised classification

Example

Attributes				label Target attribute
Name	Balance	Age	Employed	Write-off
Mike	\$200,000	42	no	yes
Mary	\$35,000	33	yes	no
Claudio	\$115,000	40	no	no
Robert	\$29,000	23	yes	yes
Dora	\$72,000	31	no	no

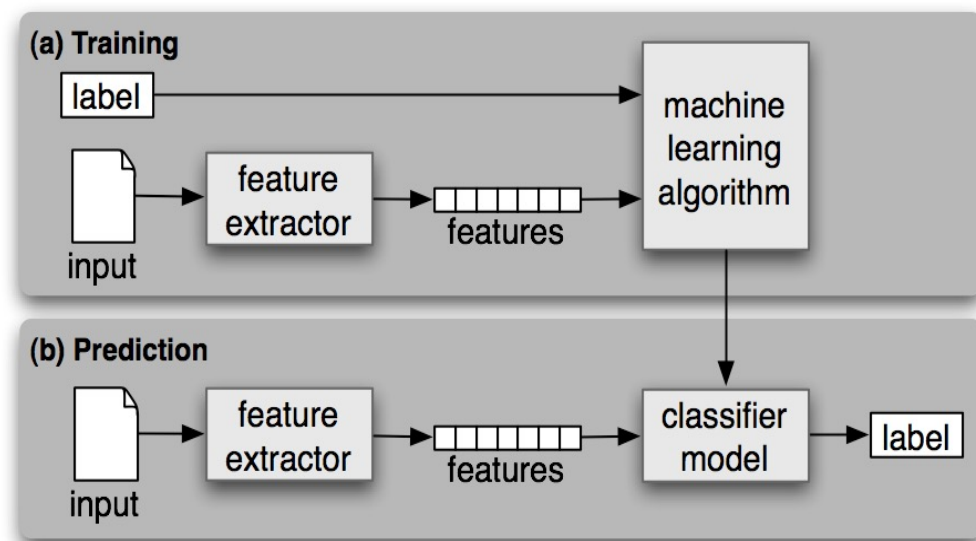
This is one row (example).
 Feature vector is: **<Claudio,115000,40,no>**
 Class label (value of Target attribute) is **no**

↑
predict class

→ นำมาสร้าง case แล้ว form knowledge มาทำ pattern แบบนี้ (generalization)
เป็นวิธีการที่ในวิชาอื่น ๆ เช่น วิทยาศาสตร์/การแพทย์/parameter/etc. หรือใช้หลักการที่ปรับมาจากวิชาอื่น ๆ มาสร้าง pattern ของสิ่งใหม่

Induction (Training)

- The creation of models from data is known as model induction (training).
- Induction is a term from philosophy that refers to generalizing from specific cases to general rules
- Our models are general rules in a statistical sense
- Most inductive procedures have variants that induce models both for classification and regression



Which attribute should be used to segment?

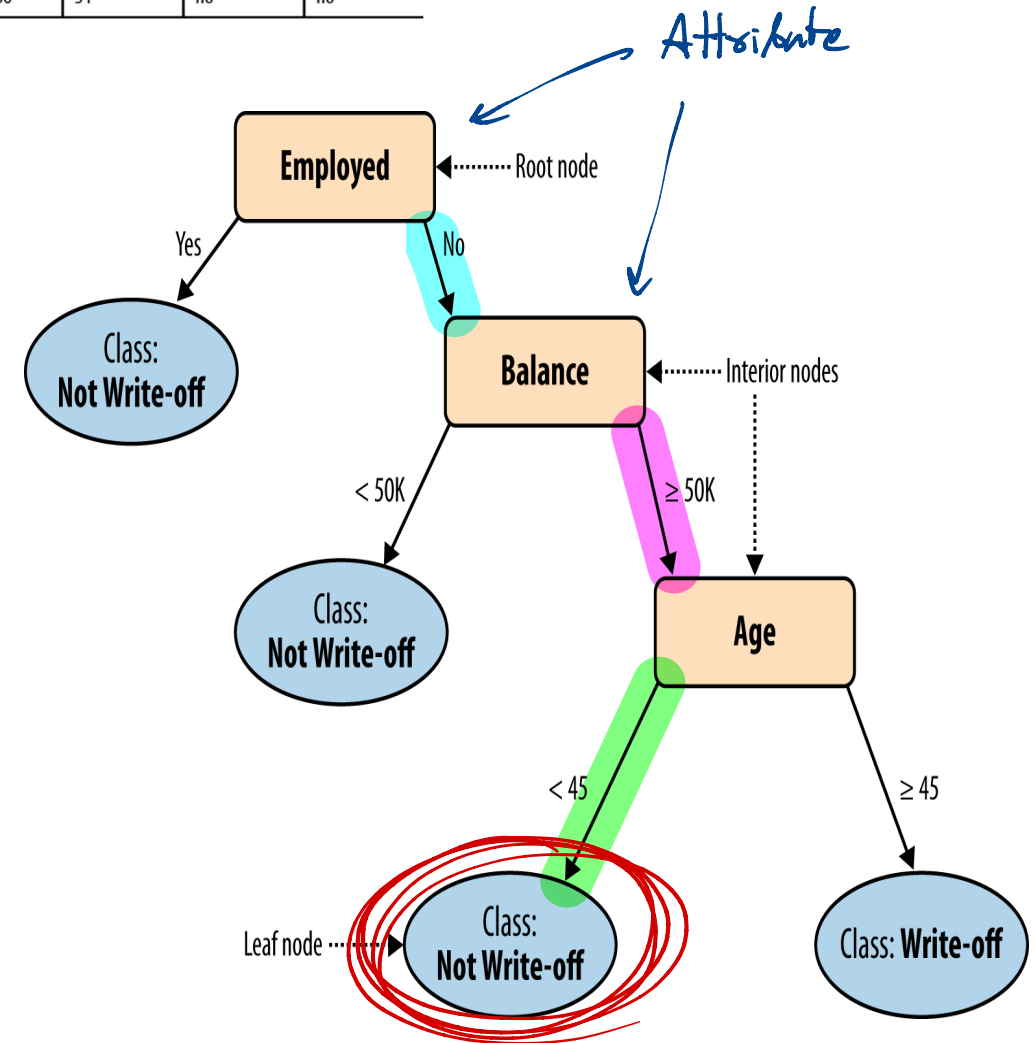
feature selection , L1 : Lasso , L2 : Ridge

- **Fundamental concept:** Which variable contains the most information?
- **Aims:** automatic selection, ranking

Decision Tree

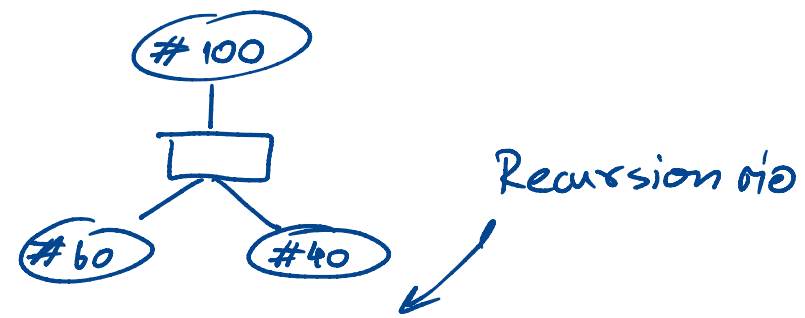
- A tree consists of nodes: interior and terminal
- Interior node contains a test of an attribute
- Terminal node is a segment

Name	Balance	Age	Employed	Write-off
Mike	\$200,000	42	no	yes
Mary	\$35,000	33	yes	no
Claudio	\$115,000	40	no	no
Robert	\$29,000	23	yes	yes
Dora	\$72,000	31	no	no



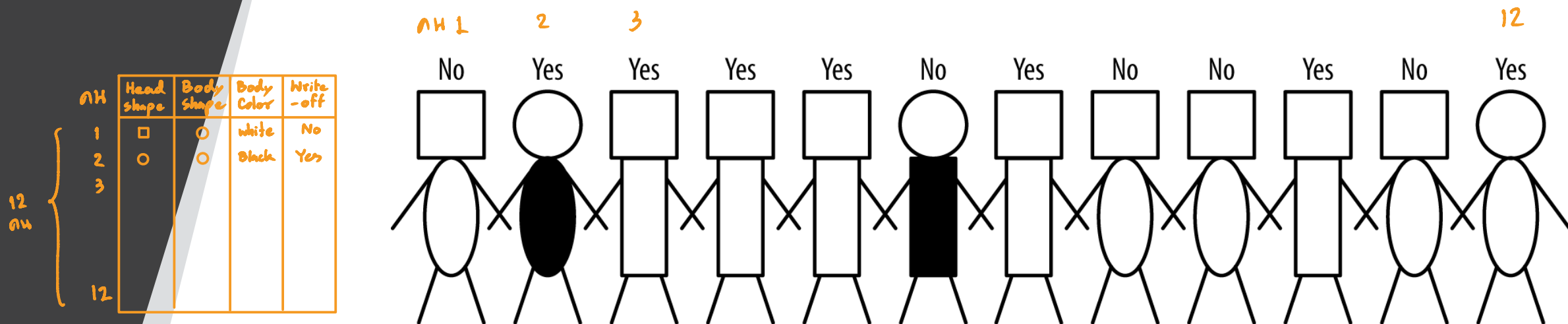
<Claudio, 115000, 40, No>?

Tree Induction



- How do we create a decision tree from data?
- Tree induction takes a divide-and-conquer approach,
 1. starting with the whole dataset
 2. applying variable selection to create subgroups
 3. Recursively repeating step 2 for each subgroup
- We will illustrate this using the write-off example

Example: Write-off



การแบ่งกลุ่ม. split data

- Attributes
 - head-shape: square, circular
 - body-shape: rectangular, oval
 - body-color: black, white
- Target variable
 - write-off: Yes, No

Which attribute should be **best** to segment these people into groups, in a way to distinguish write-off from non-write-off?

Purity

- Technically, we would like the group to be as pure as possible.
- Pure means homogeneous with respect to the target variable
- If some member in the group has a different target then the group is impure

- Comparing

קבוצה G1 סטוכסטית

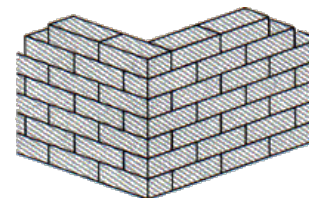
- $G1 = \{Y, Y, Y, Y\}$ דוֹמֵם

- $G2 = \{Y, N, N, Y\}$

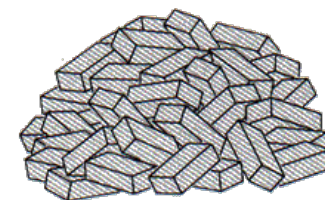
In real data, however, we rarely find pure segments.

Entropy

- Entropy is a measure of disorder that can be applied to a set, such as one of our individual segments
- Disorder corresponds to how mixed (impure) the segment is with respect to the target
- For example, a mixed up segment with lots of write-offs and lots of non-write-offs would have high entropy



(a)



(b)

$$p_1 = 0, p_2 = 1 : \text{entropy} = -0 \log(0) - 1 \log(1) = 0$$
$$p_1 = \frac{1}{2}, p_2 = \frac{1}{2} : \text{entropy} = -\frac{1}{2} \log_2\left(\frac{1}{2}\right) - \frac{1}{2} \log_2\left(\frac{1}{2}\right) = 1$$

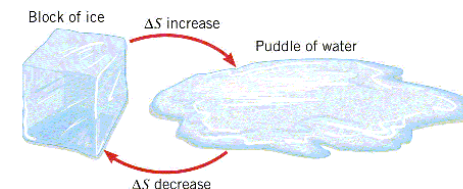
Entropy Formula

- More technically, entropy is defined as

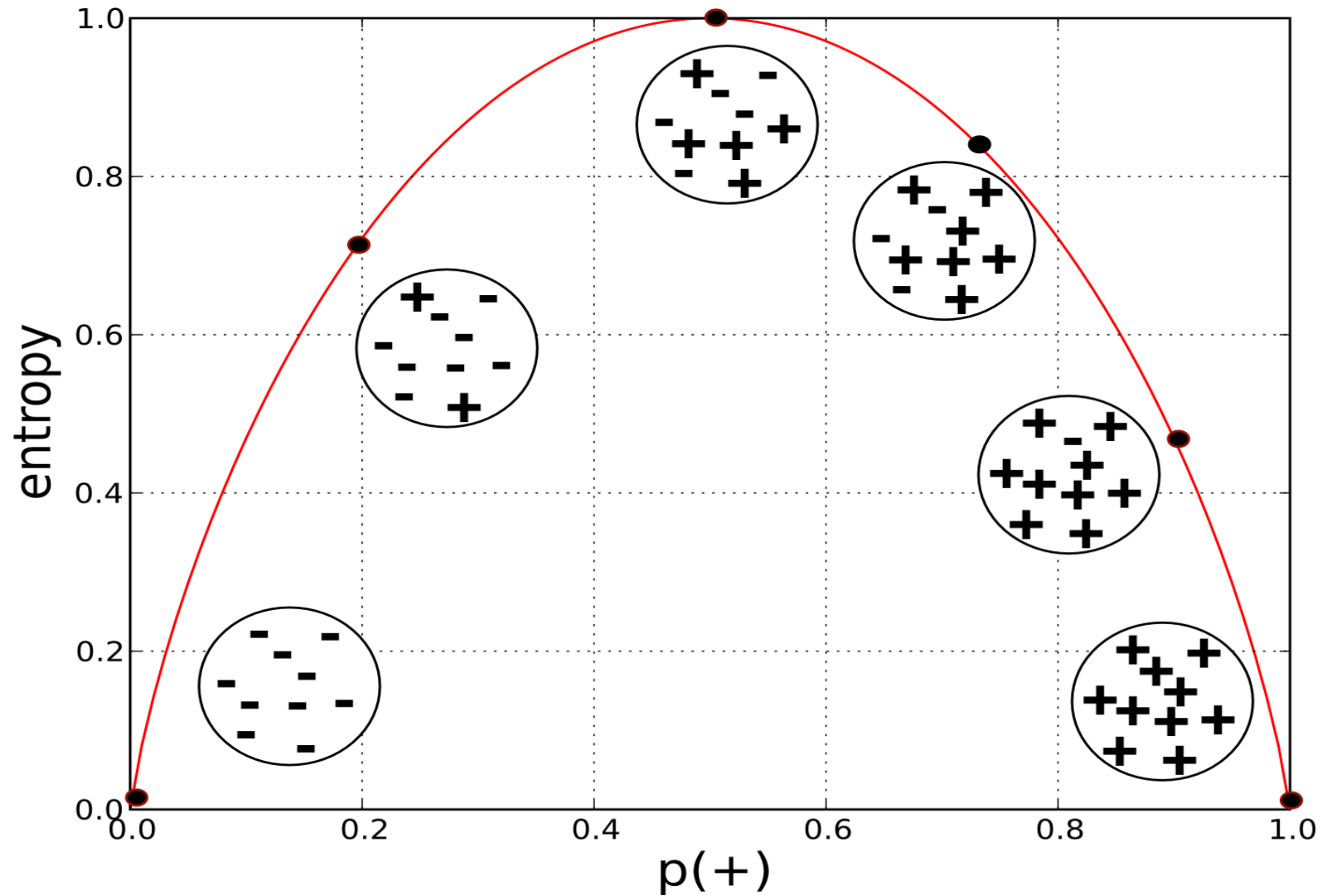
$$\text{entropy} = -p_1 \log(p_1) - p_2 \log(p_2) - \dots$$

- This is based on Gibbs entropy in thermodynamics
- Each p_i is the probability (the relative percentage) of property i (e.g. write-offs/non-write-offs) of the target.

2nd Law of Thermo: Entropy



Entropy of a two-class set



Information Gain Formula

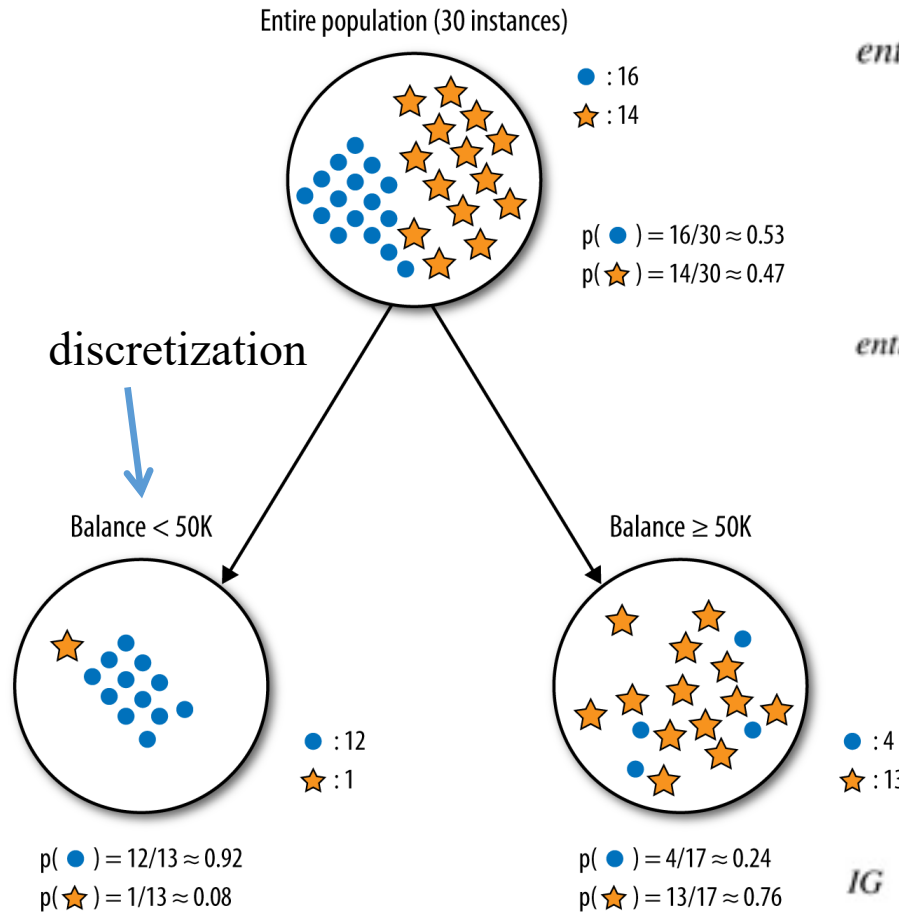
$IG(parent, children)$

$$= \text{entropy}(parent) - \text{Expected Value (ค่าคาดหวัง)} \\ [p(c_1) \times \text{entropy}(c_1) + p(c_2) \times \text{entropy}(c_2) + \dots]$$

- The entropy for each child (c_i) is weighted by the proportion of instances belonging to that child, $p(c_i)$.

(Attribute)
ตัวแปรที่เราใช้ split IG คือตัวที่เราใช้ split

Example 1



$$\begin{aligned}
 \text{entropy}(\text{parent}) &= -[p(\bullet) \times \log_2 p(\bullet) + p(\star) \times \log_2 p(\star)] \\
 &\approx -[0.53 \times (-0.9) + 0.47 \times (-1.1)] \\
 &\approx 0.99 \quad (\text{very impure})
 \end{aligned}$$

Entropy of the left child is

$$\begin{aligned}
 \text{entropy}(\text{Balance} < 50K) &= -[p(\bullet) \times \log_2 p(\bullet) + p(\star) \times \log_2 p(\star)] \\
 &\approx -[0.92 \times (-0.12) + 0.08 \times (-3.7)] \\
 &\approx 0.39
 \end{aligned}$$

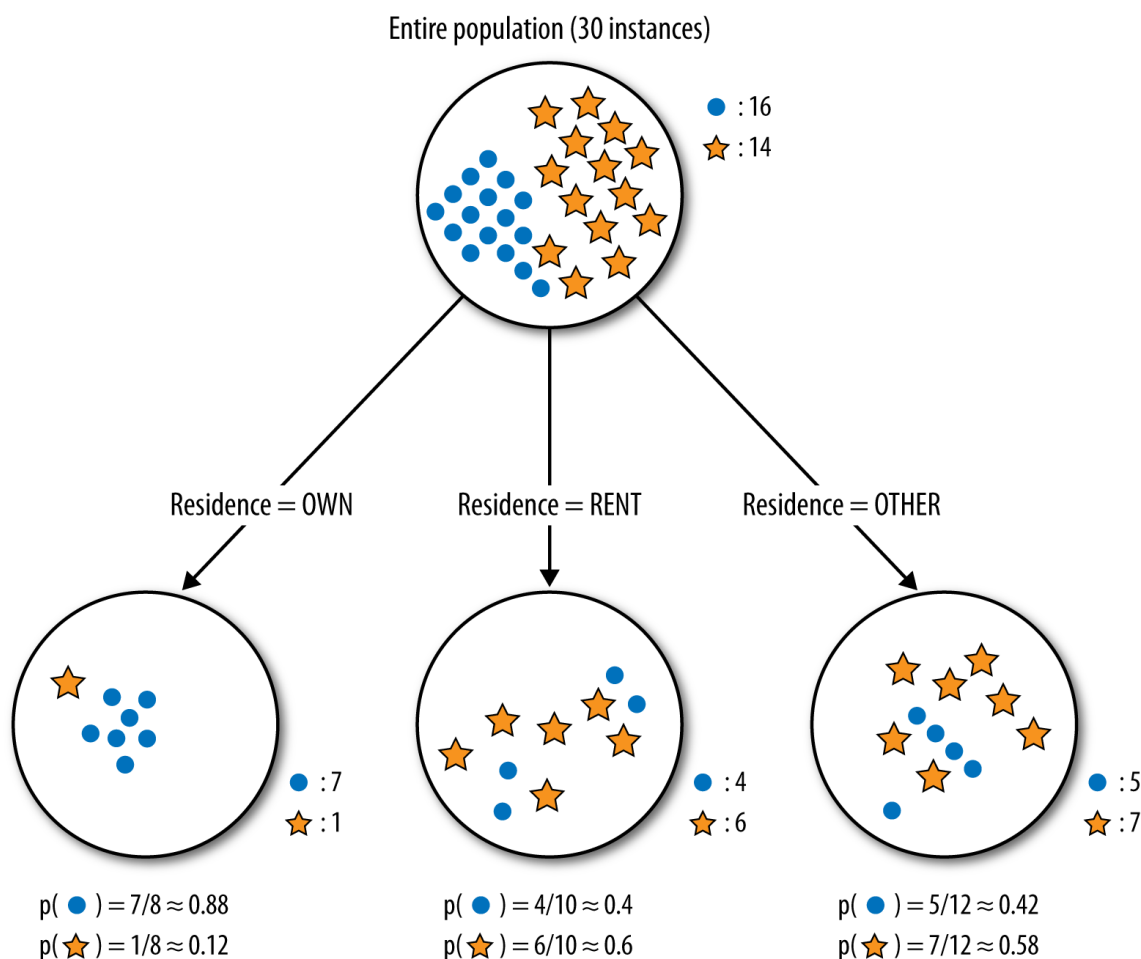
Entropy of the right child is

$$\begin{aligned}
 \text{entropy}(\text{Balance} \geq 50K) &= -[p(\bullet) \times \log_2 p(\bullet) + p(\star) \times \log_2 p(\star)] \\
 &\approx -[0.24 \times (-2.1) + 0.76 \times (-0.39)] \\
 &\approx 0.79
 \end{aligned}$$

Information gain is

$$\begin{aligned}
 IG &= \text{entropy}(\text{parent}) - [p(\text{Balance} < 50K) \times \text{entropy}(\text{Balance} < 50K) \\
 &\quad + p(\text{Balance} \geq 50K) \times \text{entropy}(\text{Balance} \geq 50K)] \\
 &\approx 0.99 - [0.43 \times 0.39 + 0.57 \times 0.79] \\
 &\approx 0.37
 \end{aligned}$$

Example 2



Calculations are omitted

$$\text{entropy}(\text{parent}) \approx 0.99$$

$$\text{entropy}(\text{Residence}=\text{OWN}) \approx 0.54$$

$$\text{entropy}(\text{Residence}=\text{RENT}) \approx 0.97$$

$$\text{entropy}(\text{Residence}=\text{OTHER}) \approx 0.98$$

$$\text{IG} \approx 0.13$$

Residence variable is less informative than Balance.

Splitting criteria

ใช้สูตร $\log$ ให้น่าสนใจ :: ค่าไม่ stable เมื่อเข้าใกล้ infinity หรือ $(\log(0))$

Regression: residual sum of squares

$$RSS = \sum_{\text{left}} (y_i - y_L^*)^2 + \sum_{\text{right}} (y_i - y_R^*)^2$$

where

y_L^* = mean y-value for left node

y_R^* = mean y-value for right node

Since the probabilities in a node always add up to 1

Equation 6-1. Gini impurity

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

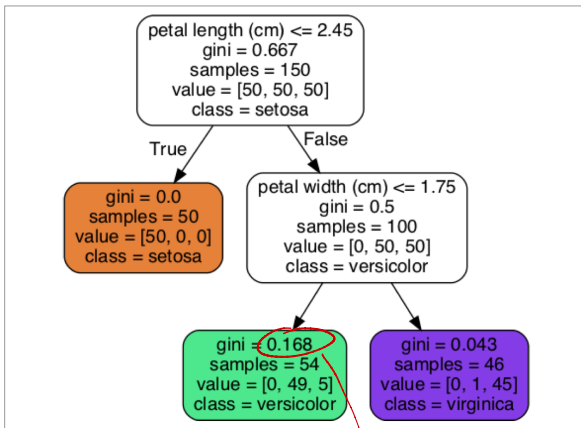


Figure 6-1. Iris Decision Tree

Classification: Gini criterion (Similar to Information Gain)

$$Gini = N_L \sum_{k=1, \dots, K} p_{kL} (1 - p_{kL}) + N_R \sum_{k=1, \dots, K} p_{kR} (1 - p_{kR})$$

where

p_{kL} = proportion of class k in left node

p_{kR} = proportion of class k in right node

$$1 - (0/54)^2 - (49/54)^2 - (5/54)^2 \approx 0.168$$

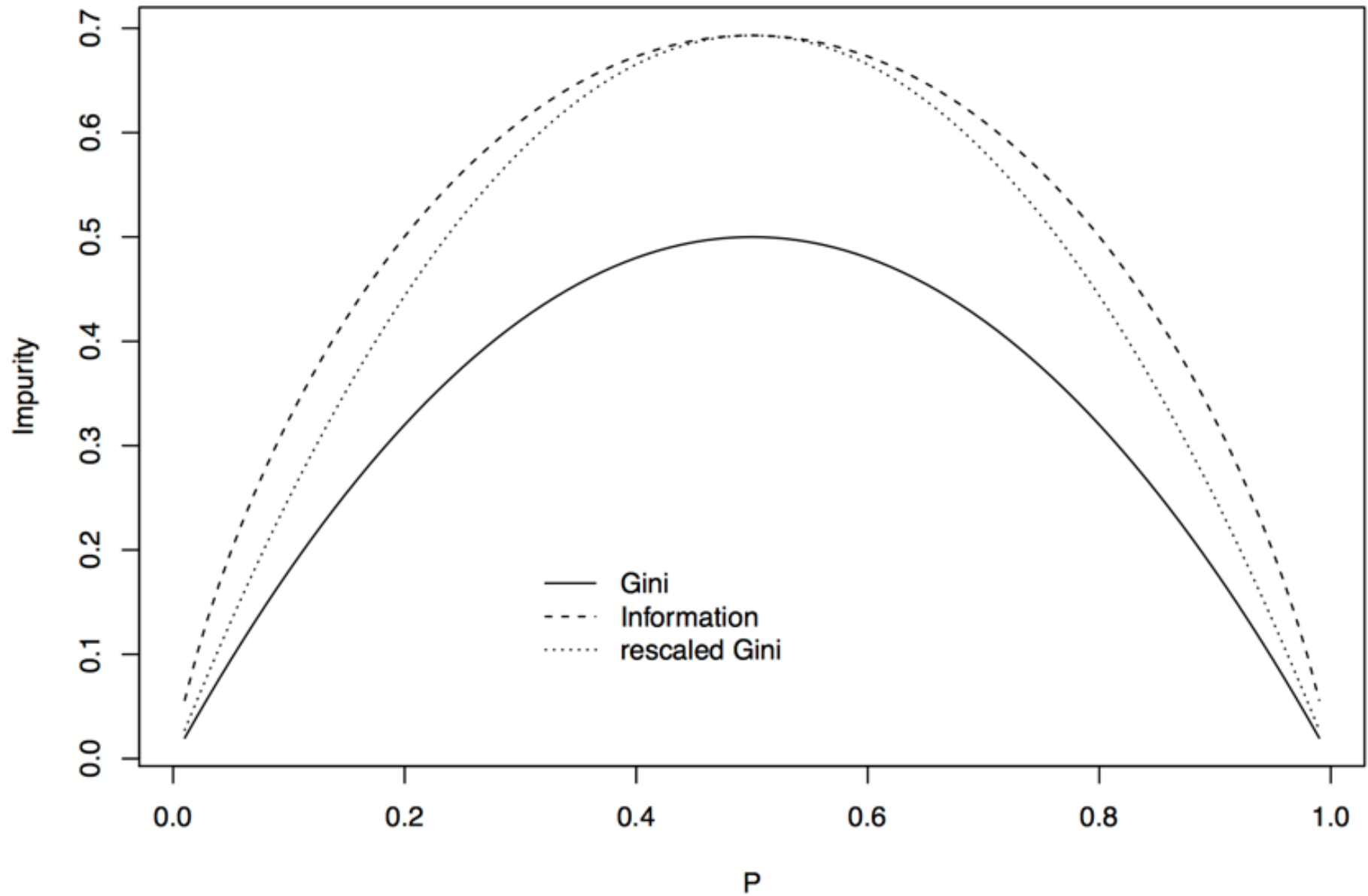
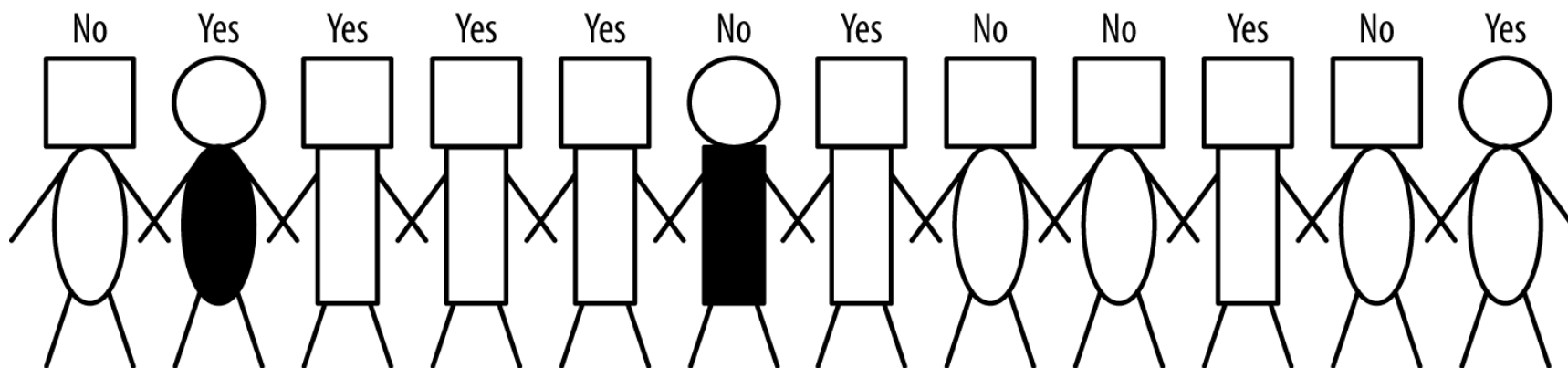


Figure 2: Comparison of Gini and Information impurity for two groups.

N : Y
5 : 7

Example



Attributes

HS head-shape: square, circular

BS body-shape: rectangular, oval

BC body-color: gray, white

Target variable

write-off: Yes, No

(หัวและตัว)
N : Y
4 : 5

(หัวและตัว)
N : Y
1 : 2

← ครบทุกเงื่อนไข

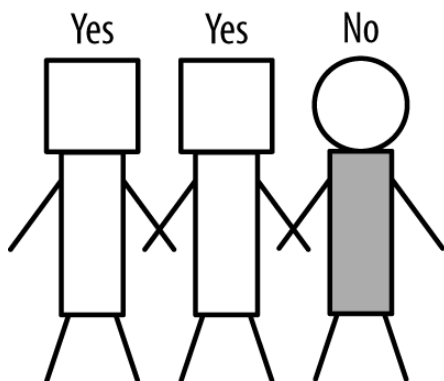
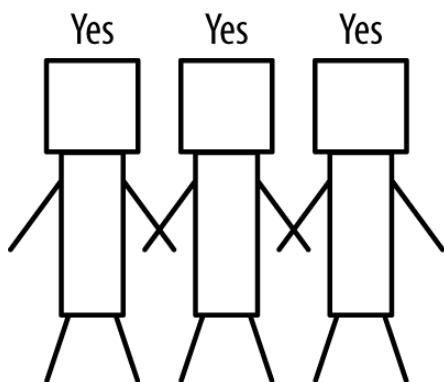
1:5

4:2

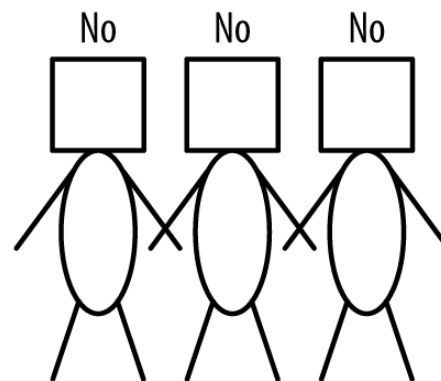
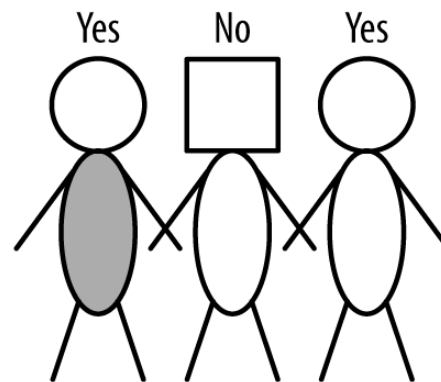
partition 6:5

First Partitioning: body-shape

Rectangular Bodies



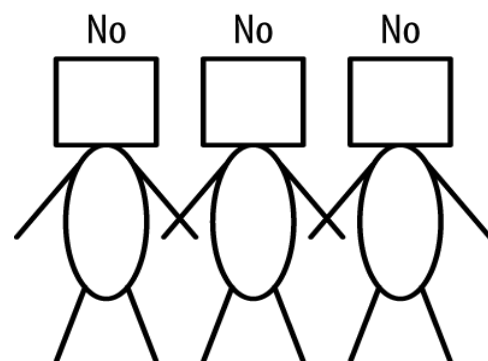
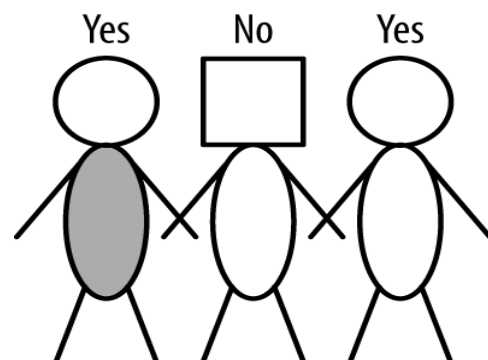
Oval Bodies



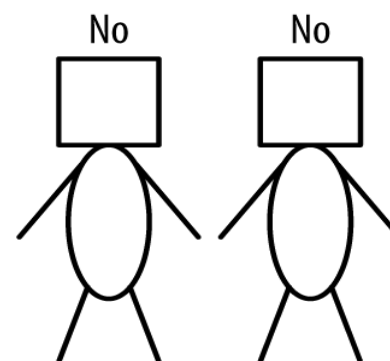
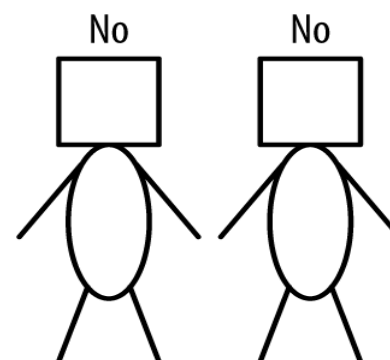
body-shape has the highest IG, so it is selected as the first attribute

2nd partitioning: oval-body, head-type

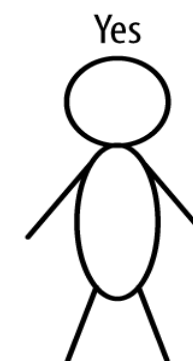
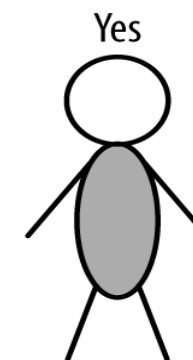
Oval Bodies



**Oval Body and
Square Head**

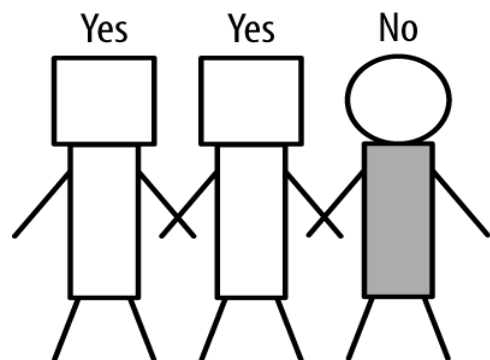
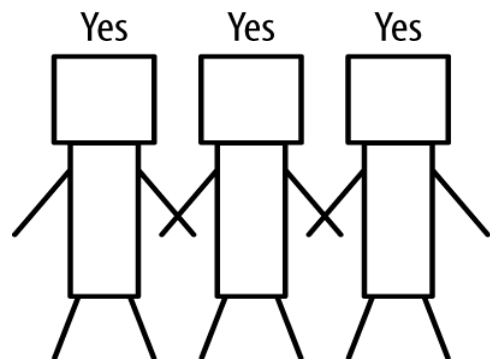


**Oval Body and
Circular Head**

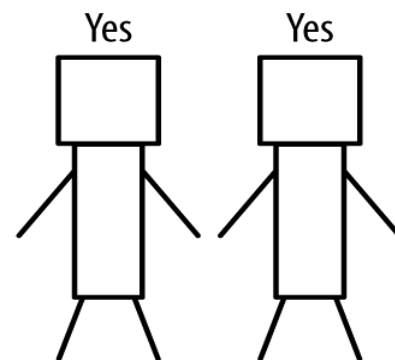
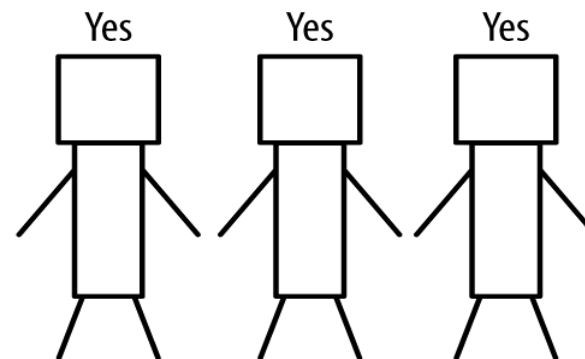


3rd partitioning: rectangular-body, body-color

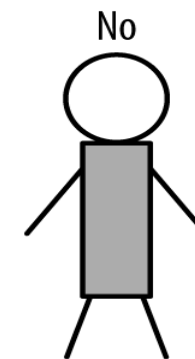
Rectangular Bodies



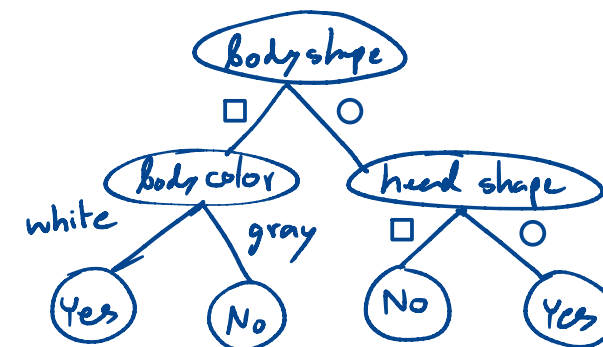
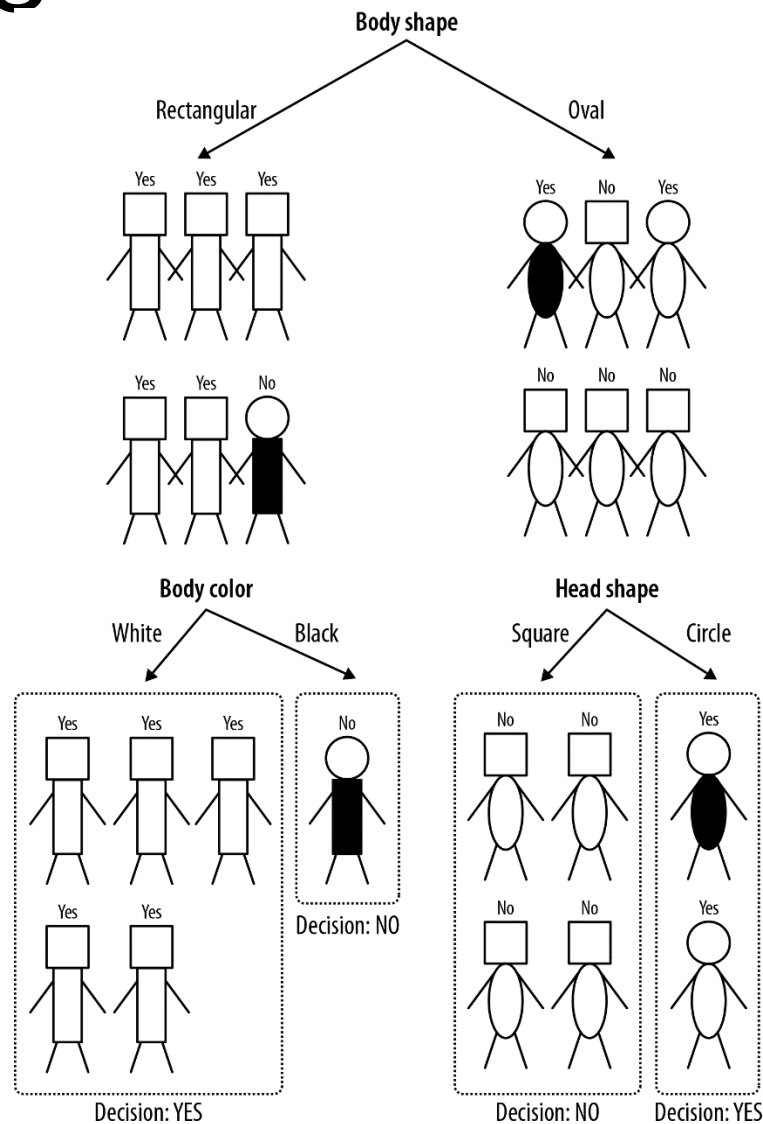
Rectangular Body
and White



Rectangular Body
and Gray



Resulting Decision Tree

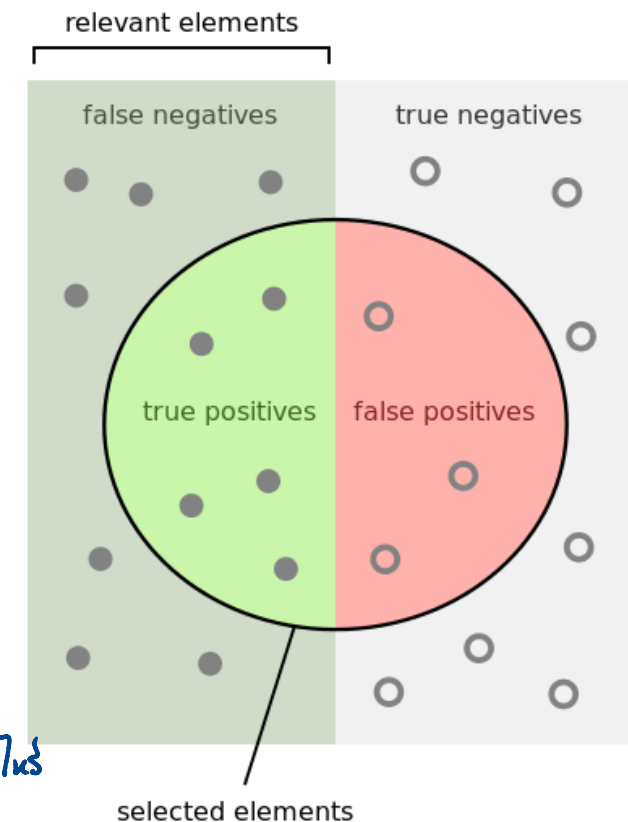


Accuracy, Precision and Recall

	Actual Positive (p)	Actual Negative (n)
The model says “Yes” = positive (y)	True positives	False positives
The model says “No” = not positive (n)	False negatives	True negatives

- Accuracy = $(TP + TN) / (TP + FP + TN + FN)$
- Recall (Completeness) = true positive rate = $TP / (TP + FN)$ *ស្រុកចំណុចវិជ្ជា*
- Precision (Exactness) = the accuracy over the cases predicted to be positive, $TP / (TP + FP)$
- F-measure = the harmonic mean of precision and recall
 = the balance between recall and precision

$$= 2 \cdot \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$



How many selected items are relevant?

Precision = $\frac{\text{true positives}}{\text{true positives} + \text{false positives}}$

How many relevant items are selected?

Recall = $\frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$

Receiver operating characteristics

Area under the ROC curve

True Positive Rate (TPR) is a synonym for recall and is therefore defined as follows:

$$TPR = \frac{TP}{TP + FN}$$

False Positive Rate (FPR) is defined as follows:

$$FPR = \frac{FP}{FP + TN}$$

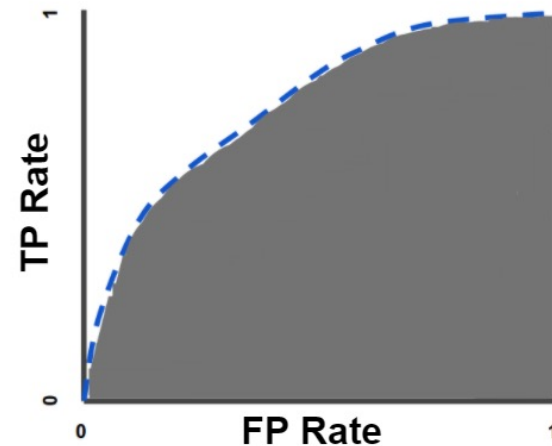


Figure 5. AUC (Area under the ROC Curve).

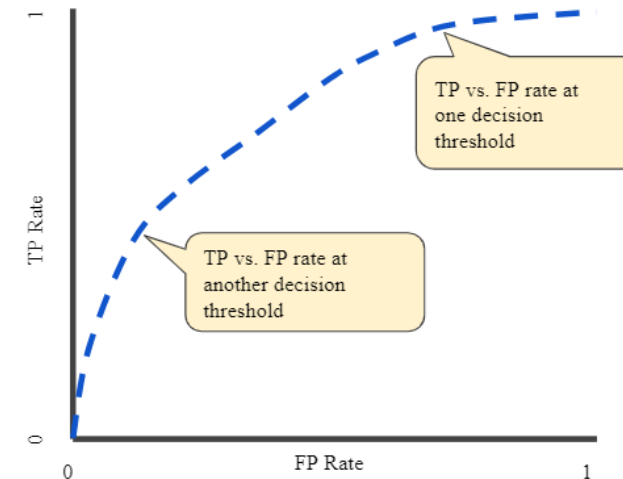
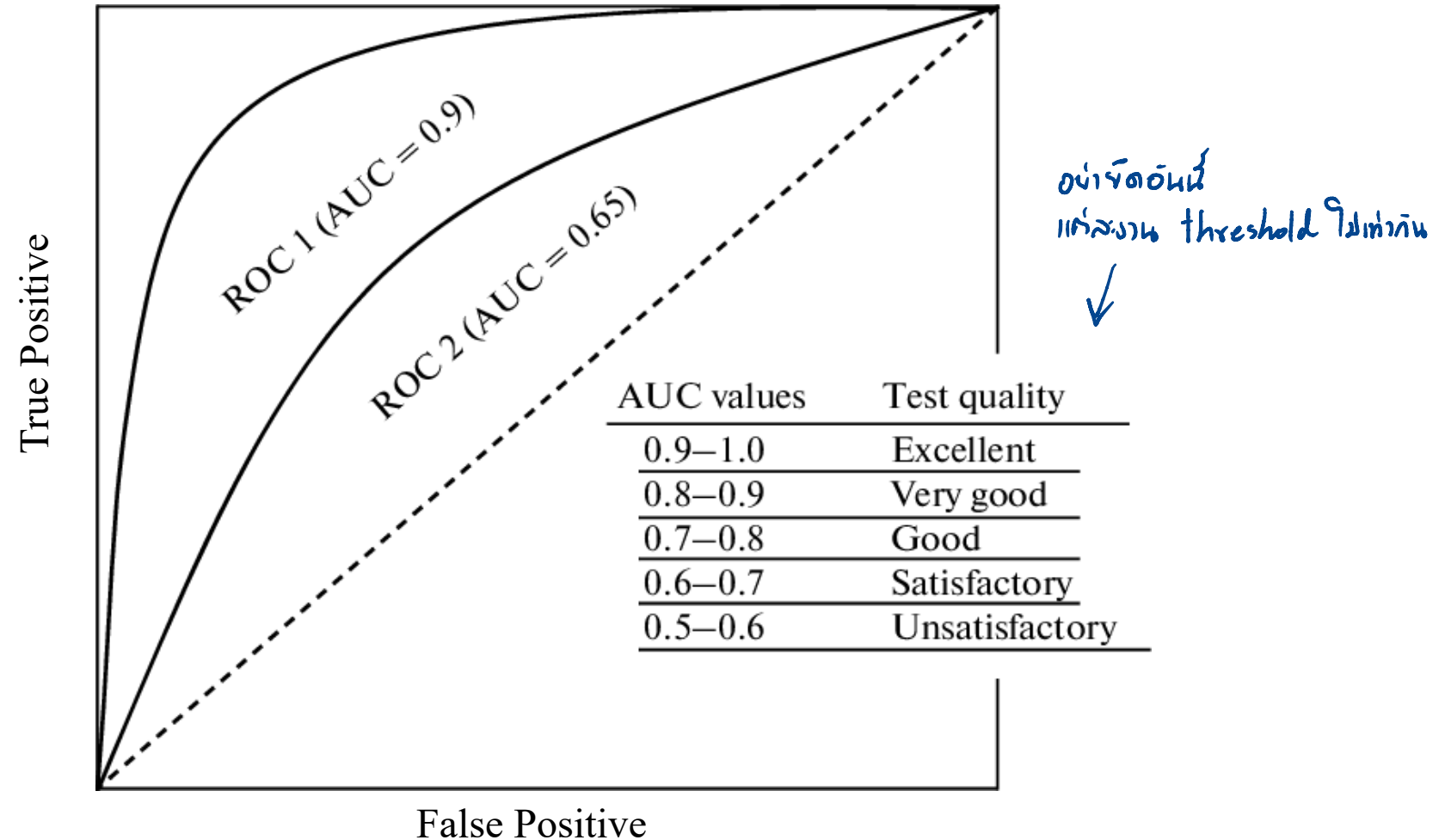


Figure 4. TP vs. FP rate at different classification thresholds.

AUC - ROC





Ensemble Models

<https://drive.google.com/file/d/1sp2quqCmxkzy2ksoHCqWgNdJImAjRrsx/view?usp=sharing>



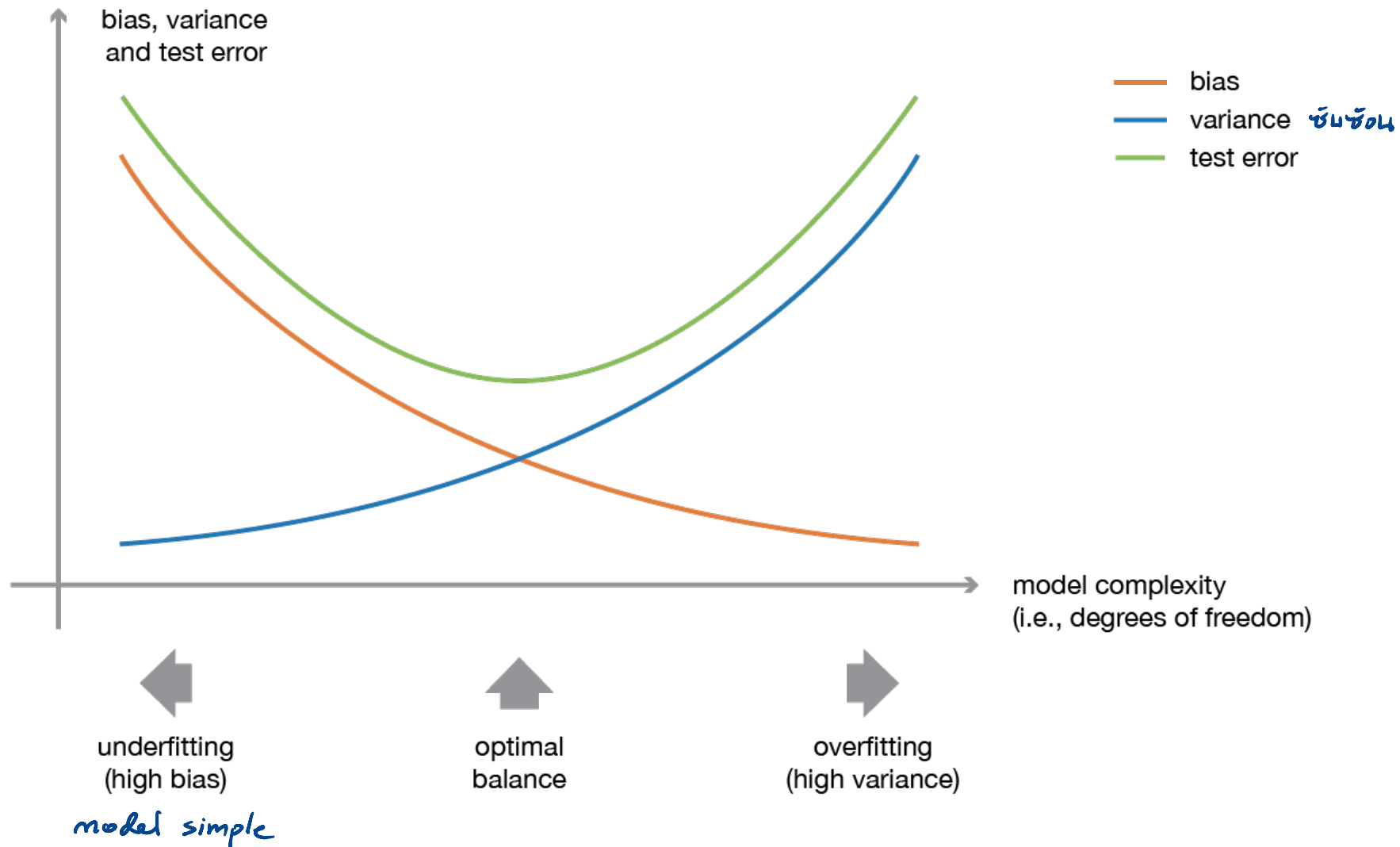
Classification and Regression Trees

Disadvantages

- *Accuracy* - current methods, such as support vector machines and ensemble classifiers often have 30% lower error rates than CART.
- *Instability* – if we change the data a little, the tree picture can change a lot. So the interpretation is not as straightforward as it appears.
- We can do better!

Random Forests

Bias variance tradeoff



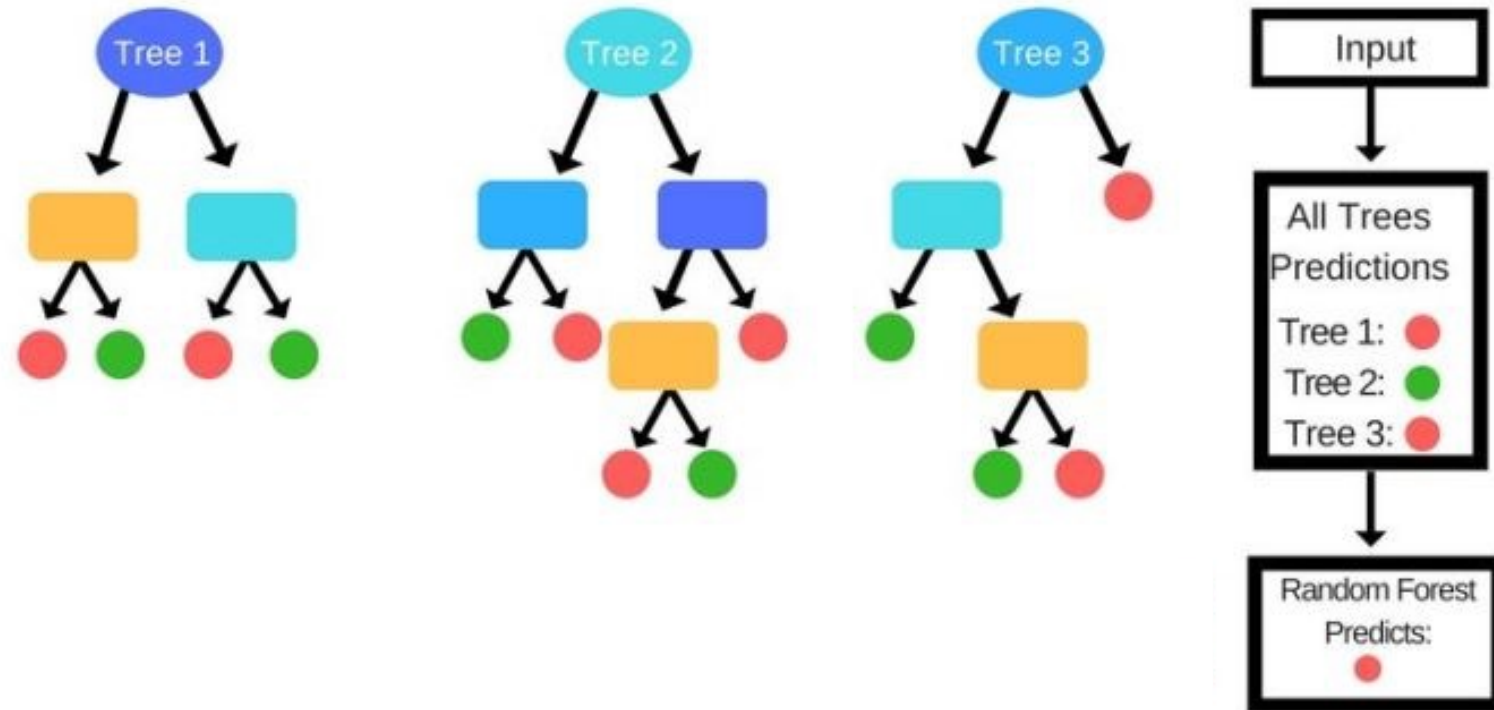
High Variance model is Variance, Bias is:

Random Forest

for regression and classification



Random Forest



[illegible]

- dataset: 100 Col
100k row
- 10k 50
1 tree (1 pattern)
500000000 data
- 10k 50
1 tree
- :

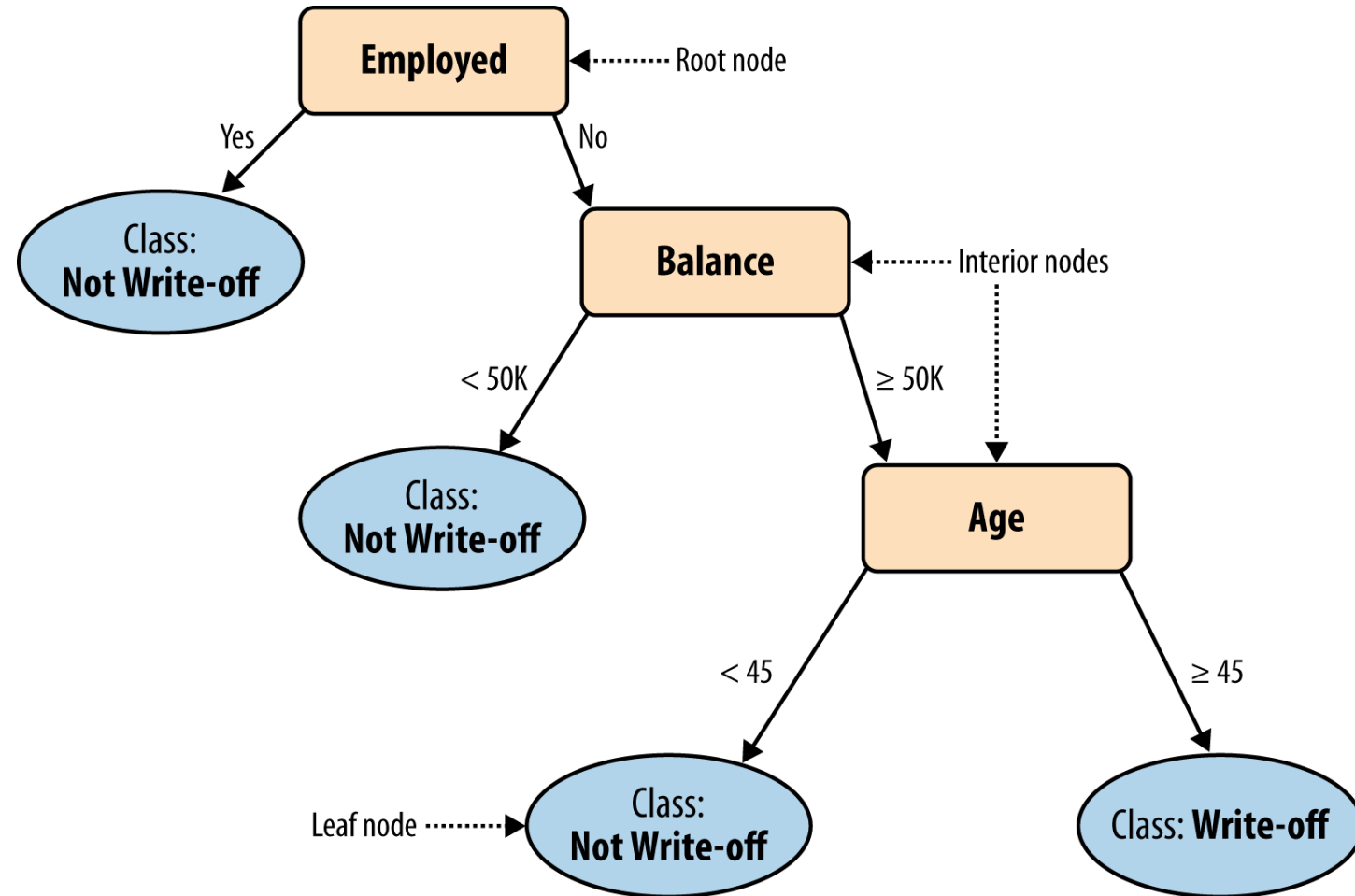
Classification and Regression Trees Pioneers

Pioneers:

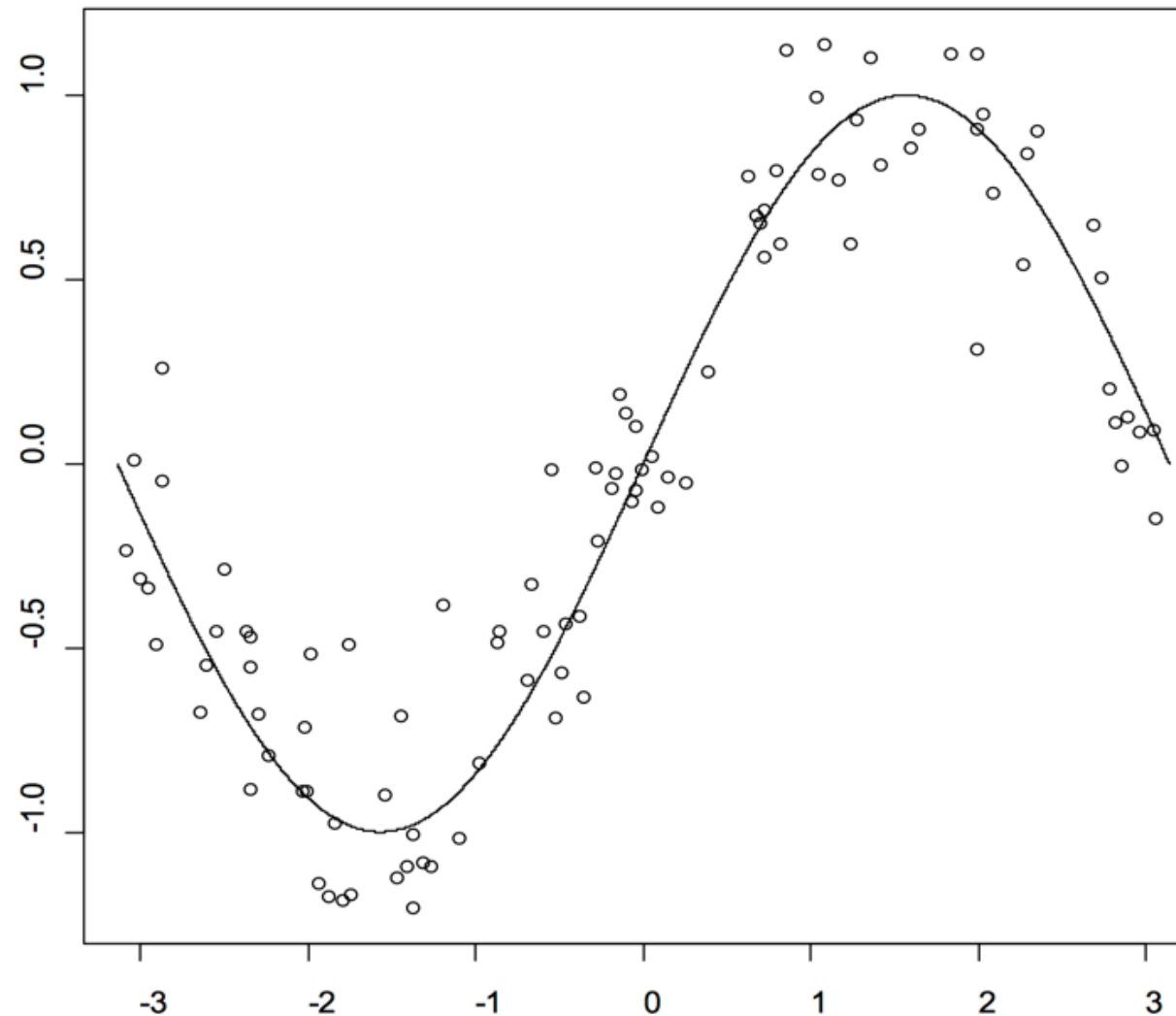
- Morgan and Sonquist (1963).
- **Breiman, Friedman, Olshen, Stone (1984). *CART***
- Quinlan (1993). *C4.5*



A Classification Tree



Data and Underlying Function

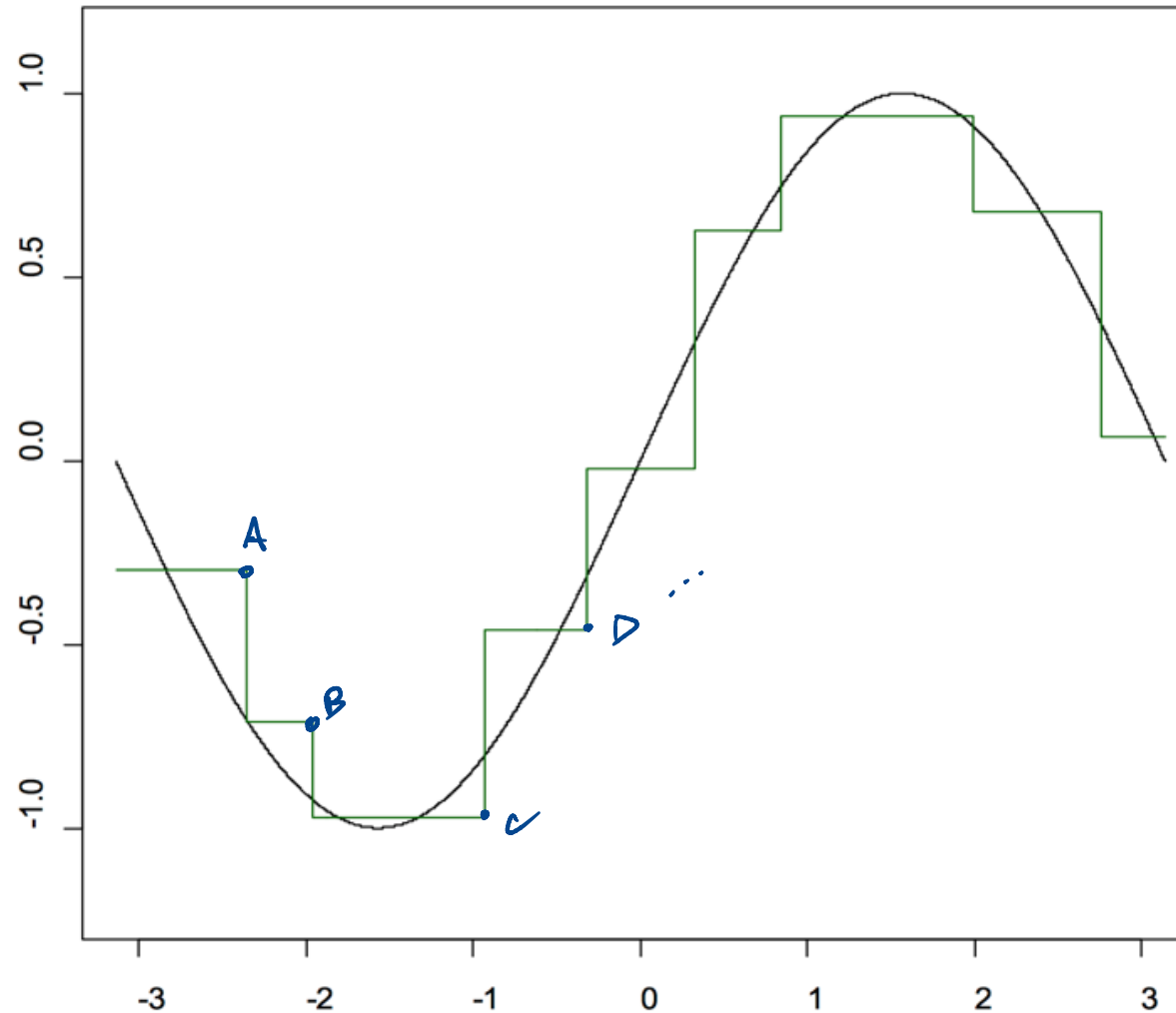
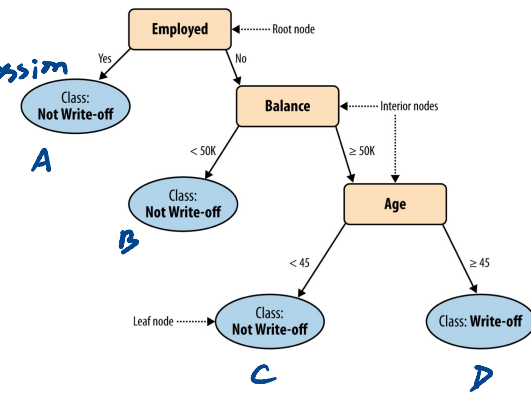


107 Decision tree and Regression

Single Regression Trees

leaf node

regression

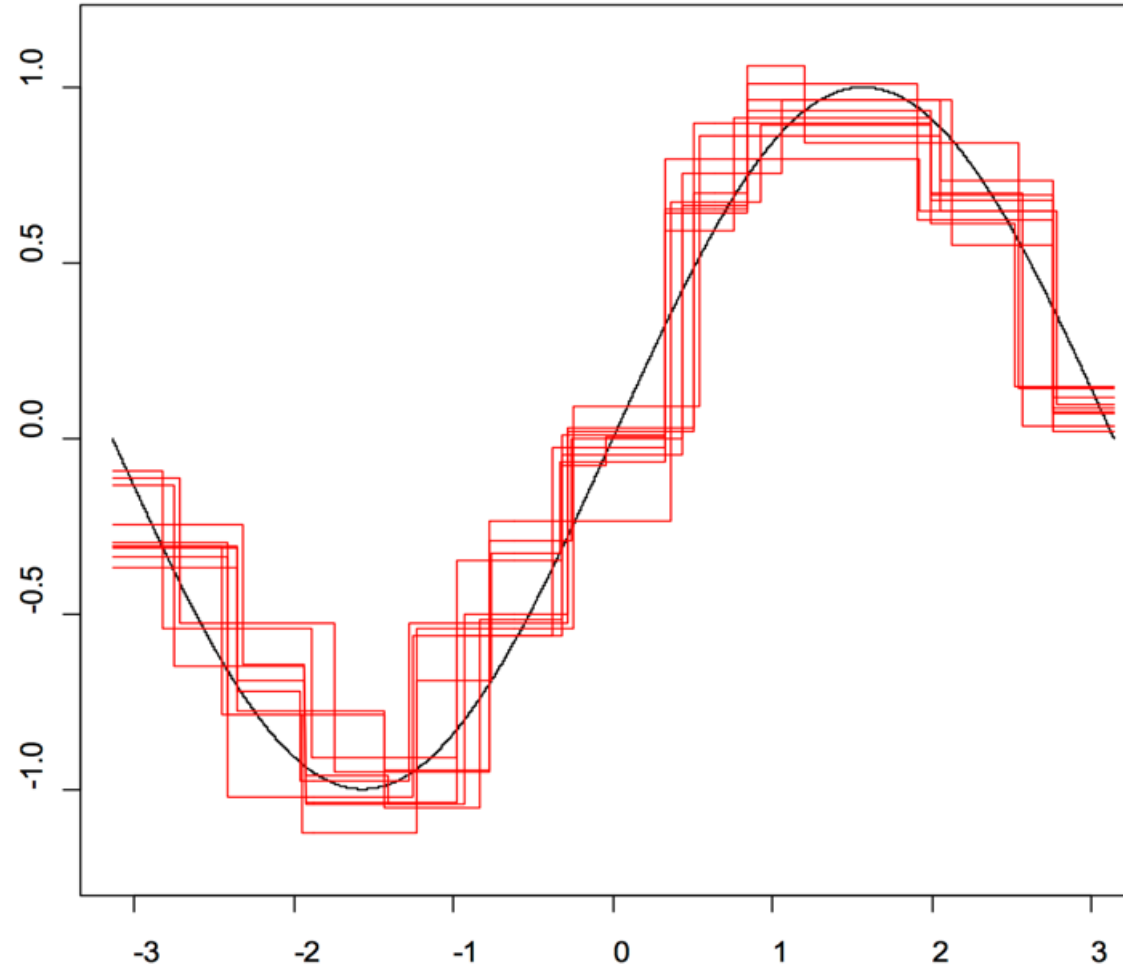


regression error

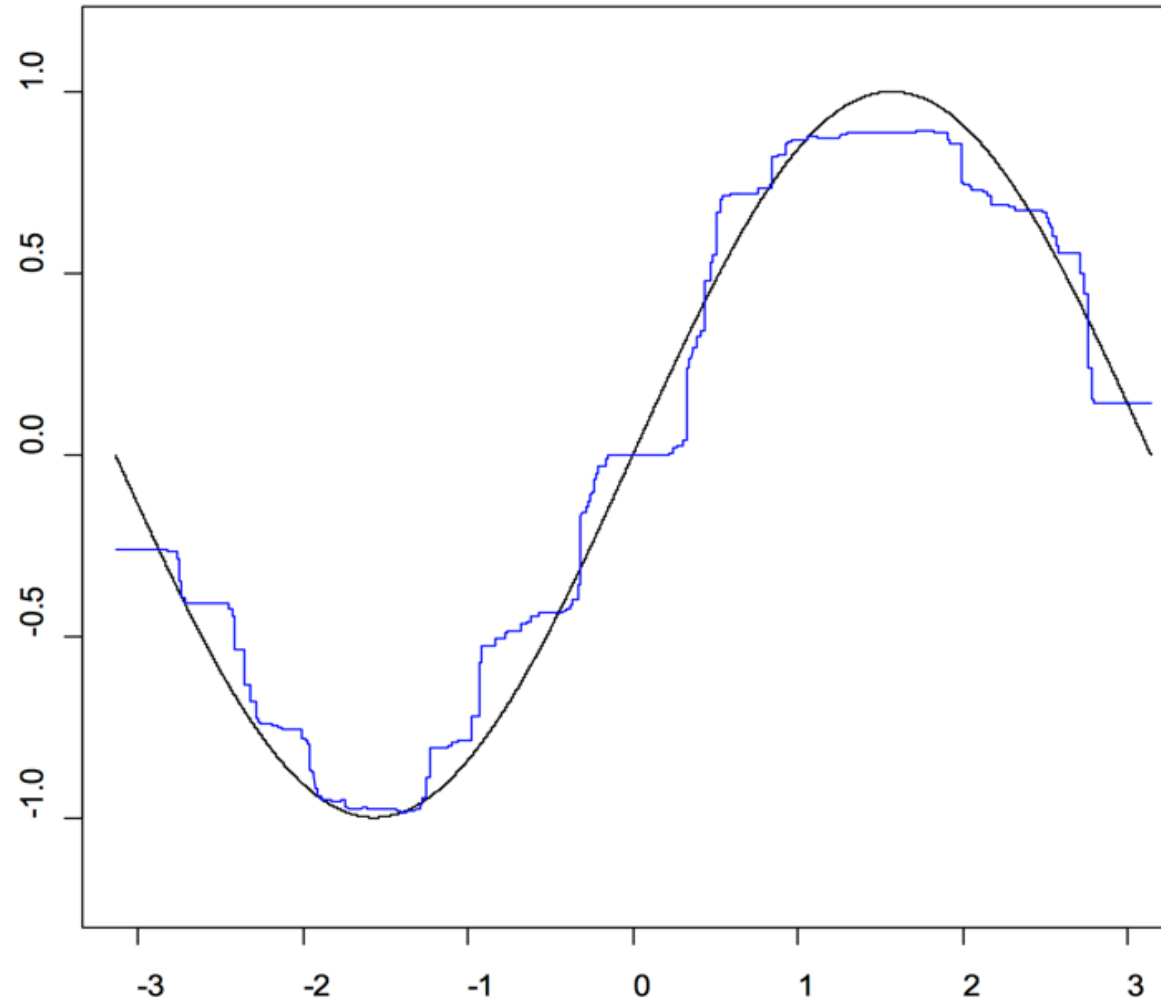
Error 140!

10 Regression Trees

๑๐ ต้น

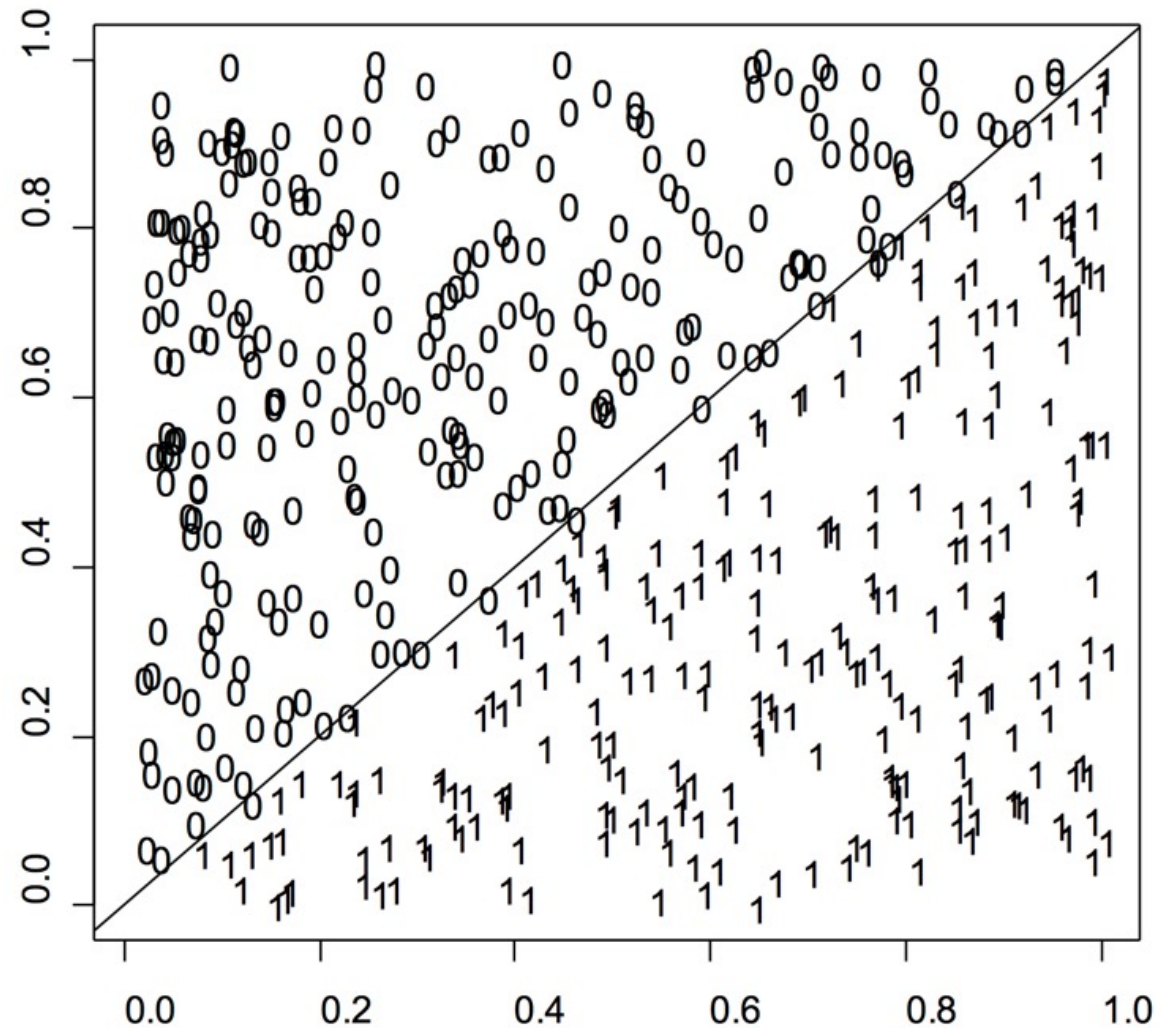


Average of 100 Regression Trees



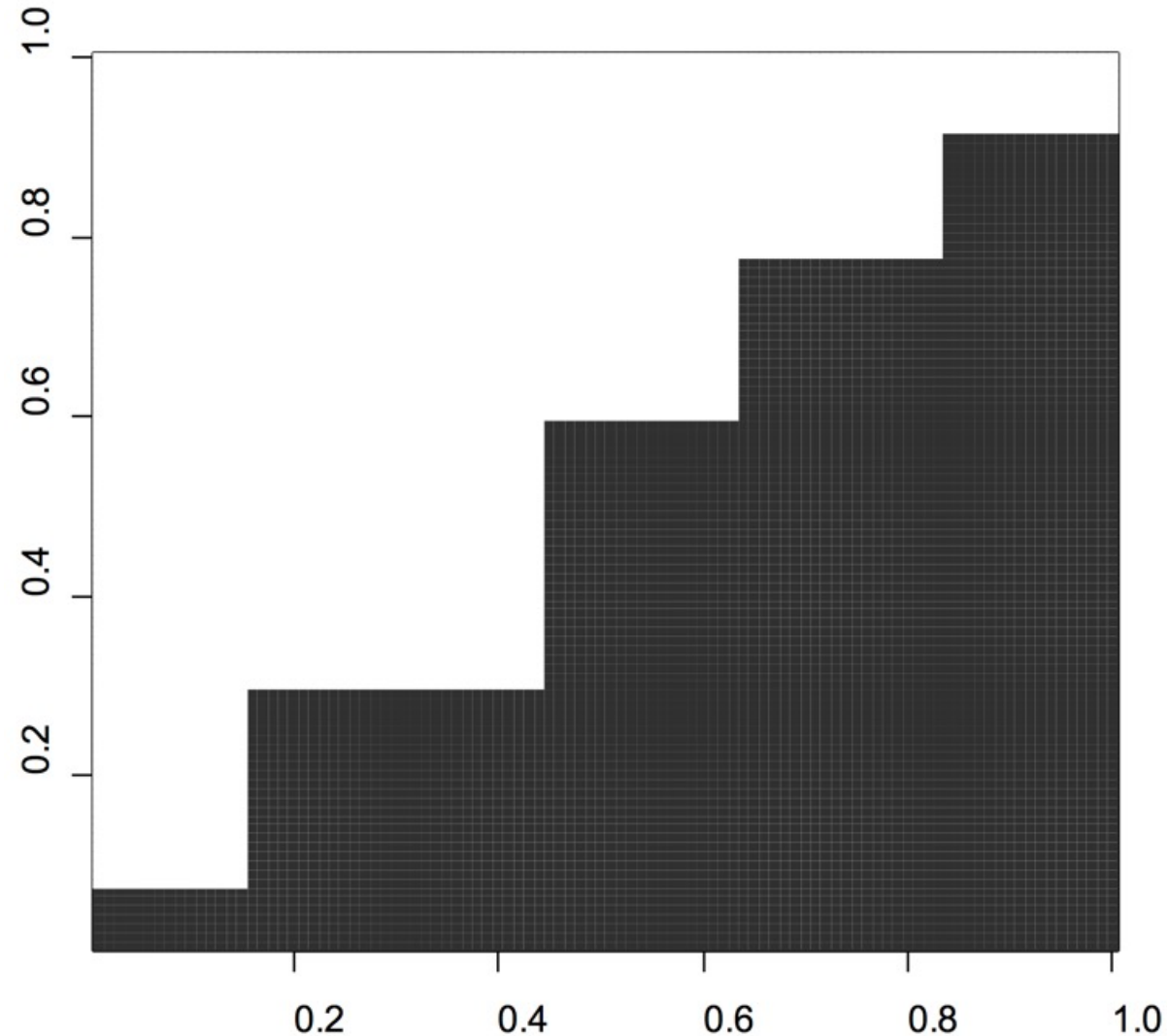
၁၀၀ ကို
လျှောက်

Hard problem for a single tree



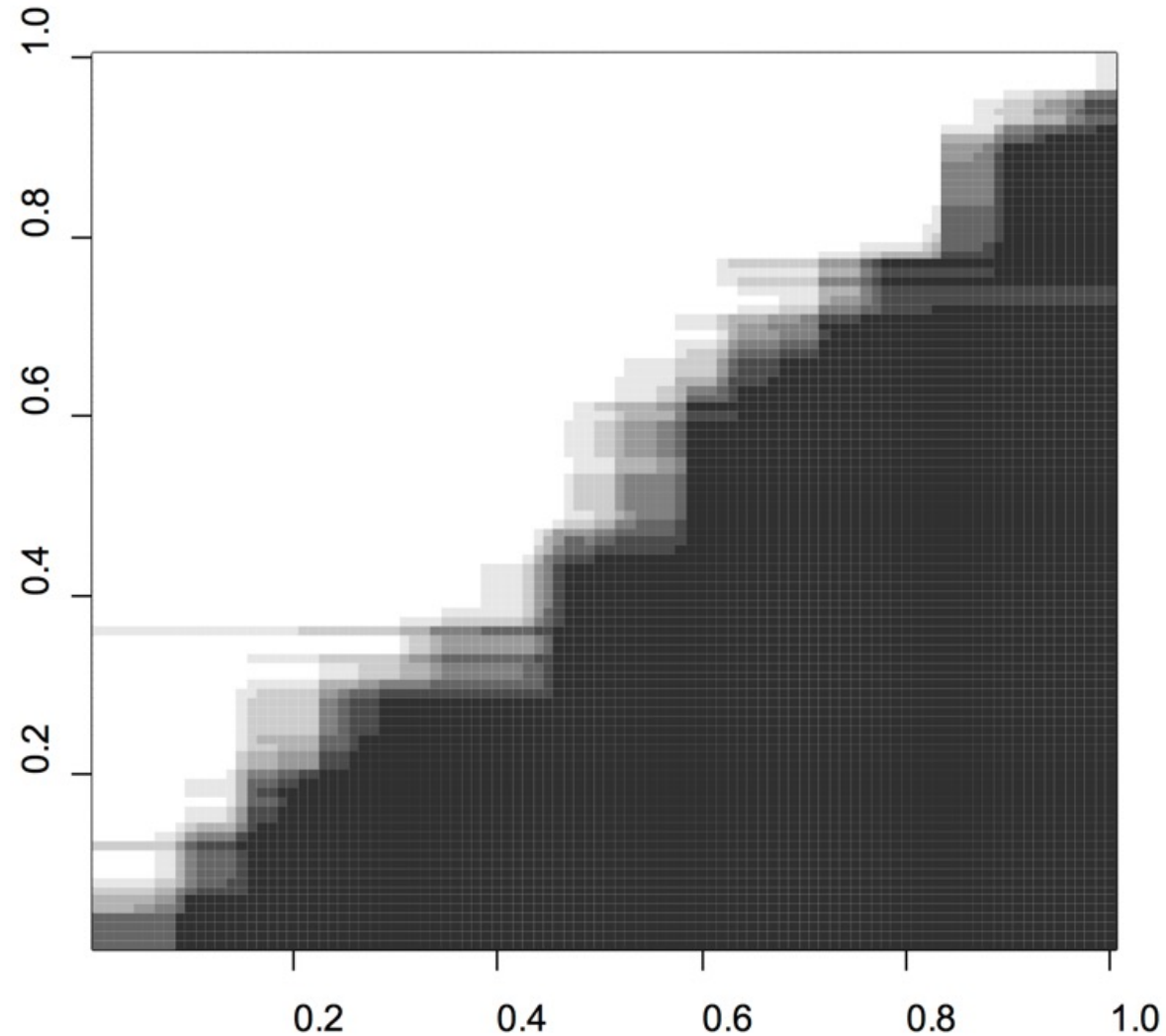
101 Decision Tree and Classification

Single Tree

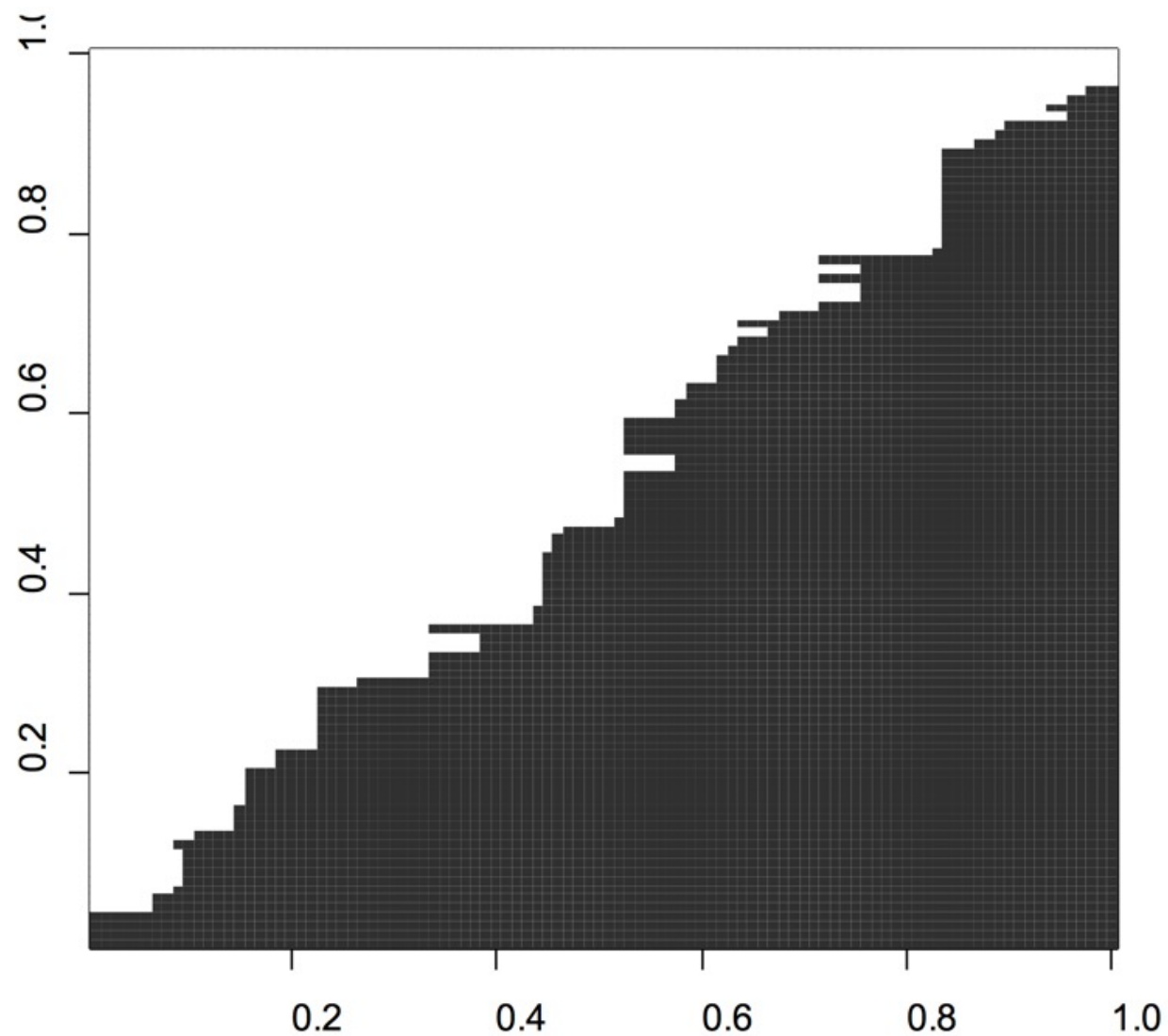


25 Averaged Trees

Boundary is Blue



25 Voted Trees



|| คำว่า: data

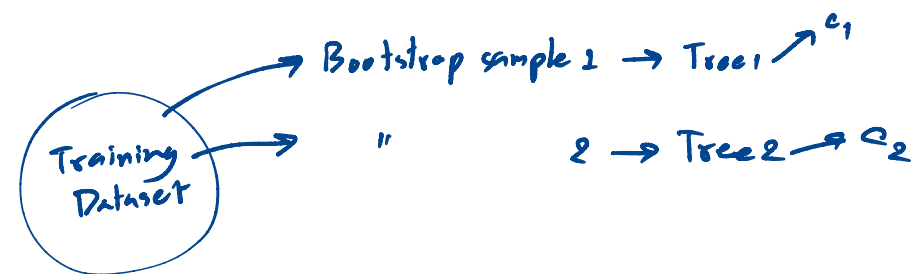
Bagging (Bootstrap Aggregating)

|| คำว่า Replacement (ซ้ำแบบสุ่ม/สลับ)

- Fit classification or regression models to bootstrap samples from the data and combine by voting (classification) or averaging (regression)

Bootstrap sample $\rightarrow f_1(x)$ Bootstrap sample $\rightarrow f_2(x)$ Bootstrap sample $\rightarrow f_3(x)$

...

Bootstrap sample $\rightarrow f_M(x)$ Combine $f_1(x), \dots, f_M(x) \rightarrow f(x)$ $f_i(x)$'s are "base learners"

Bagging (Bootstrap Aggregating)

- A bootstrap sample is chosen at random *with* replacement from the data. Some observations end up in the bootstrap sample more than once, while others are not included (“out of bag”).
၁၁၁၁: Bootstrap sample ၏ $f(x)$ ၏အားကိုးကွယ် vote ပေးမည်
 ၂၀: မပါဝင် overfit
- Bagging reduces the *variance* of the base learner but has limited effect on the *bias*.
- It's most effective if we use *strong* base learners that have very little bias but high variance (unstable). E.g. trees.

တည်ငြိမ်လွန်း linear regression :: $\downarrow$ bias, $\uparrow$ var

Random Forests

- Grow a **forest** of many trees. (100-500)
 - Grow each tree on an independent **bootstrap sample*** from the training data.
 - At each node:
 1. Select **m variables at random** out of all M possible variables (independently for each node).
 2. Find the best split on the selected **m** variables.
 - Grow the trees to maximum depth (classification).
Vote/average the trees to get predictions for new data.
- *Sample N cases at random with replacement.

Random Forests

Inherit many of the advantages of CART:

- Applicable to both regression and classification problems. Yes.
- Handle categorical predictors naturally. Yes.
- Computationally simple and quick to fit, even for large problems. Yes.
- No formal distributional assumptions (non-parametric). Yes.
- Can handle highly non-linear interactions and classification boundaries. Yes.
- Automatic variable selection. Yes. But need variable importance too.
- Handles missing values ~~through surrogate variables.~~ Using proximities.
- ~~Very easy to interpret if the tree is small.~~ NO! *သိသိအောင် သိသိသိ tree*

Random Forests

Improve on CART with respect to:

- *Accuracy* – Random Forests is competitive with the best known machine learning methods.
- *Instability* – if we change the data a little, the individual trees may change but the forest is relatively stable because it is a combination of many trees.

Two natural questions

- ***Why bootstrap? (Why subsample?)***
 - Bootstrapping → out-of-bag data →
 - Estimated error rate and confusion matrix
 - Variable importance
- ***Why trees?***
 - Trees → proximities →
 - Missing value fill-in
 - Outlier detection
 - Illuminating pictures of the data (clusters, structure, outliers)

Load data

```
[1] from google.colab import drive  
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[5] import pickle
```

```
X_train, X_test, y_train, y_test = \  
    pickle.load(open('/content/drive/MyDrive/GSB/hr_attrition.data', 'rb'))
```

```
[6] print(X_train.shape)  
print(X_test.shape)  
print(y_train.shape)  
print(y_test.shape)
```

(1029, 44)

(441, 44)

(1029,)

(441,)

Modeling - RandomForestClassifier

```
[7] from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(min_samples_leaf=20, max_depth=8, class_weight='balanced')
rf.fit(X_train,y_train)
```

▼ RandomForestClassifier

```
RandomForestClassifier(class_weight='balanced', max_depth=8,
                        min_samples_leaf=20)
```

Random Forest – Feature Importance

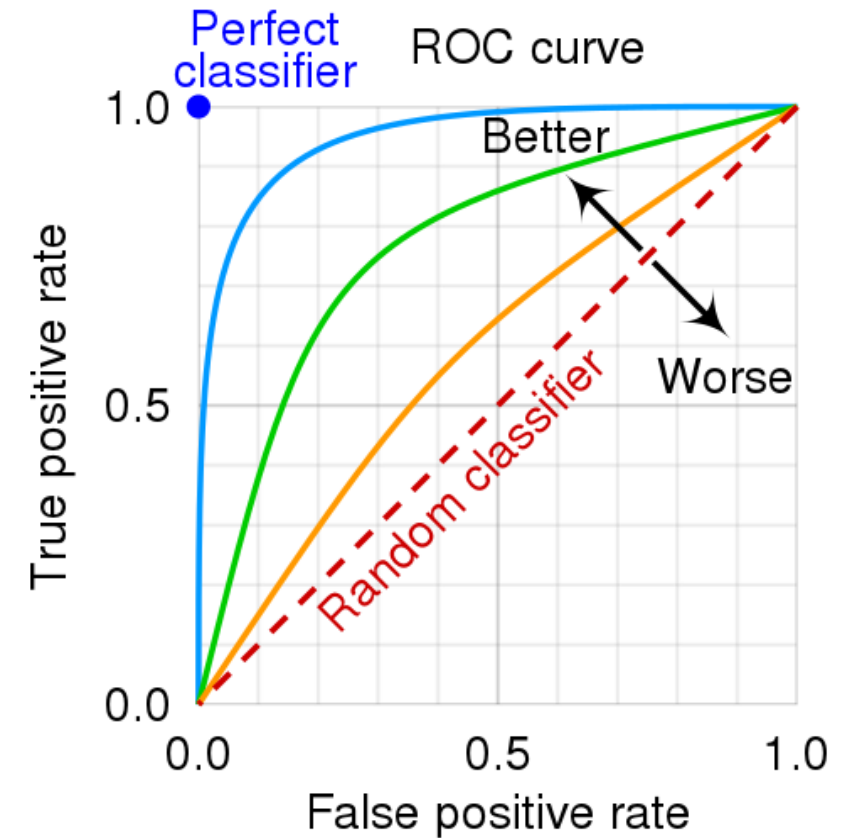
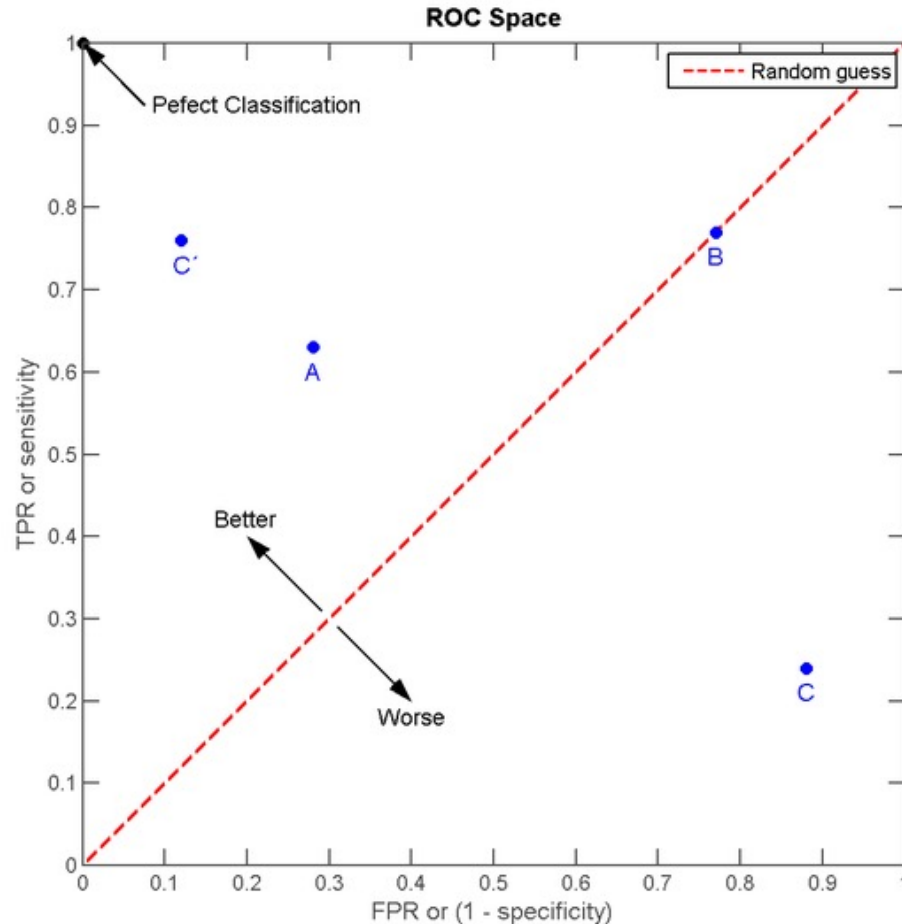
```
[9] import pandas as pd

pd.DataFrame(dict(Feature=rf.feature_names_in_,
                  Value=rf.feature_importances_))\
    .sort_values(by='Value', ascending=False)\
    .head(20)
```

	Feature	Value
30	MonthlyIncome	0.111079
20	OverTime_Yes	0.088303
40	YearsAtCompany	0.065036
36	StockOptionLevel	0.063221
21	Age	0.057085
22	DailyRate	0.045814



Receiver Operating Characteristics



Evaluation

AUC = 0.82

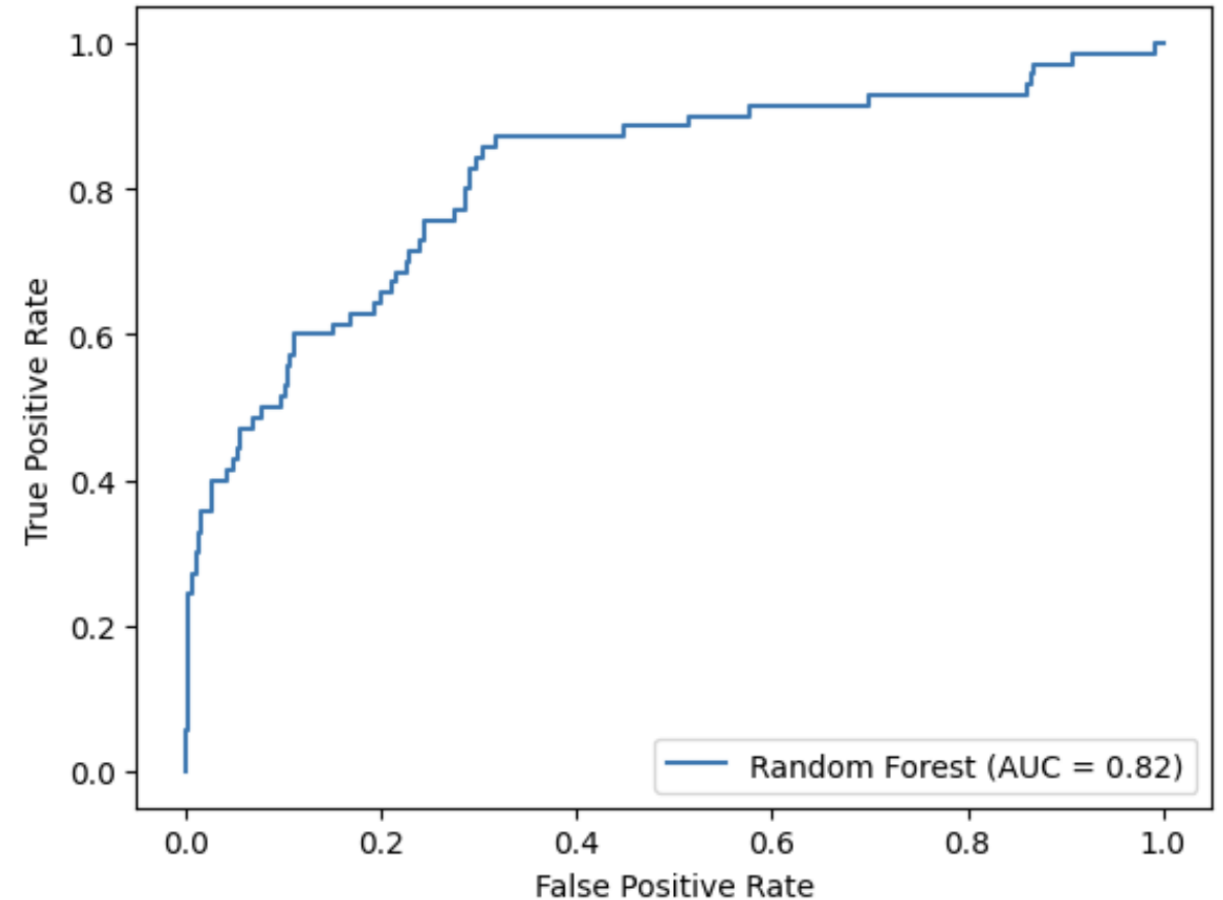


```
from sklearn.metrics import RocCurveDisplay, roc_curve, auc

y_res = rf.predict_proba(X_test)
fpr, tpr, thresholds = roc_curve(y_test, y_res[:,1])
roc_auc = auc(fpr, tpr)
display = RocCurveDisplay(fpr=fpr, tpr=tpr, roc_auc=roc_auc,
                          estimator_name='Random Forest')

display.plot()
```

<sklearn.metrics._plot.roc_curve.RocCurveDisplay at 0x7f2470e9aee0>



Bagging vs Boosting

pattern ตรวจจับ

- Bagging (random forests) perform well for multi-class object detection, which tends to have a lot of statistical noise. *data is noisy*
- Gradient Boosting performs well when you have unbalanced data such as in real time risk assessment.

skew ความเป็นเหตุเป็นผล class ที่ไม่สมดุล



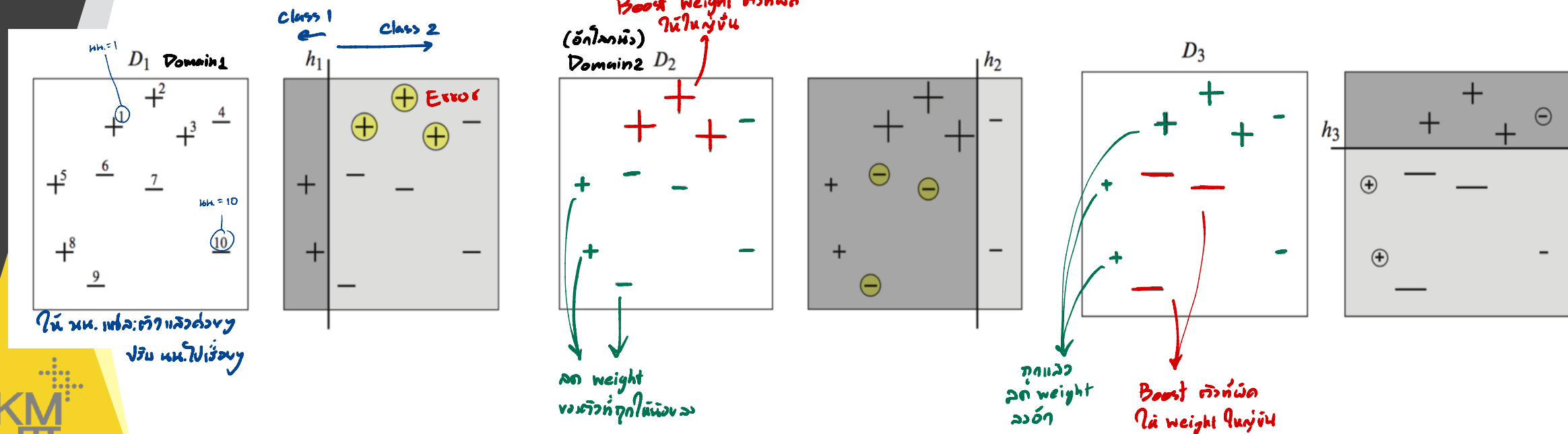
Boosting



AdaBoost (Schapire and Freund, 2012)

- Fit additive model $\sum_t c_t h_t(x)$

weight c_t หนัก: คำนวณ
- In each stage, introduce a **weak learner** to compensate the shortcoming of existing weak learner



AdaBoost



การรวม weak classifier

$$H(x) = \sum_t \rho_t h_t(x)$$

$$H = \text{sign} \left(0.42 \begin{array}{|c|c|} \hline \text{shaded} & \text{white} \\ \hline \end{array} + 0.65 \begin{array}{|c|c|} \hline \text{shaded} & \text{white} \\ \hline \end{array} + 0.92 \begin{array}{|c|c|} \hline \text{shaded} & \text{white} \\ \hline \end{array} \right)$$

$$= \begin{array}{|c|c|c|c|} \hline \text{shaded} & + & + & - \\ \hline + & - & + & - \\ \hline + & - & - & - \\ \hline \end{array}$$

Gradient Boosting (9652)

- Gradient boosting = gradient descent + boosting
- Fit additive model $\sum_t c_t h_t(x)$
- In each stage, introduce a weak learner to compensate the shortcoming of existing weak learner
- In Gradient Boosting, “shortcomings” are identified by gradients. (residual)
- Both high-weight data points and gradients tell us how to improve our model.

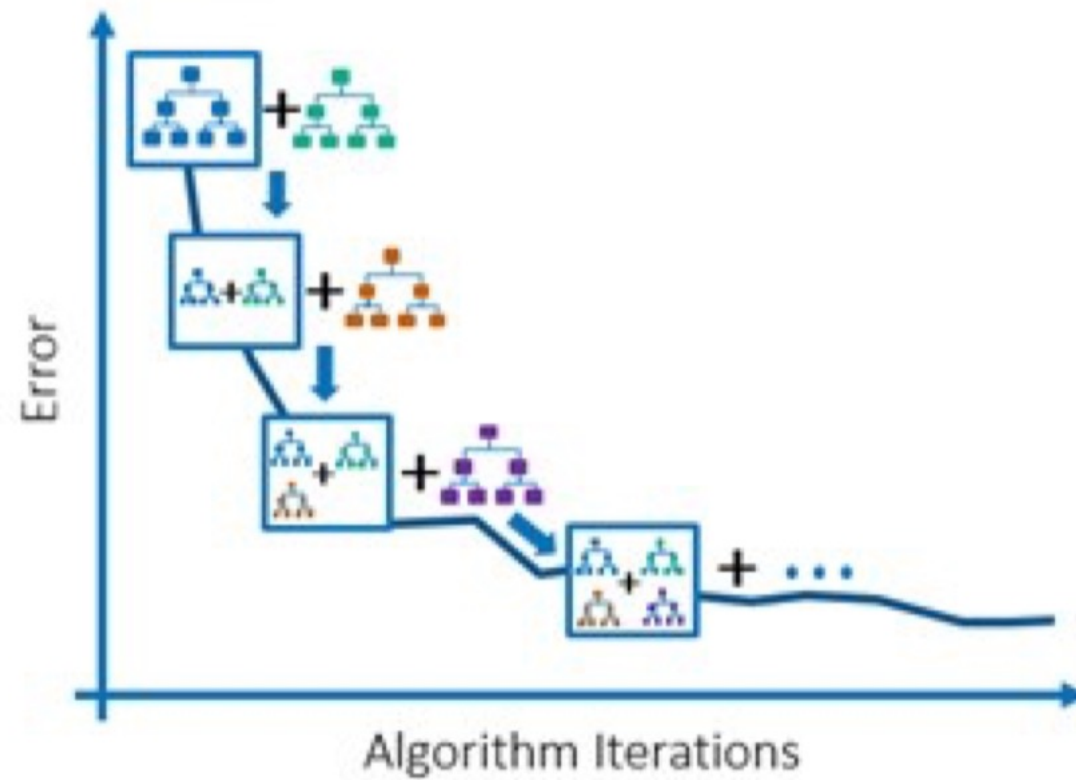
Gradient

- Gradient is calculated from the derivative of the loss function.

$$\text{Loss}(y_i, \hat{y}) = (y_i - \hat{y})^2$$

$$-\nabla_{\text{Loss}}(\hat{y}) = 2 \cdot (y_i - \hat{y})$$

Gradient boosting



Gradient boosting algorithm

Algorithm 2 Gradient boosting.

let F_0 be a “dummy” constant model

for $m = 1, \dots, M$

for each pair (x_i, y_i) in the training set

 compute the *pseudo-residual* $R(y_i, F_{m-1}(x_i)) = \text{negative gradient of the loss}$

 train a regression sub-model h_m on the pseudo-residuals

 add h_m to the ensemble: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$

return the ensemble F_M

Modeling – GradientBoostingClassifier

```
[13] from sklearn.ensemble import GradientBoostingClassifier  
  
gbc = GradientBoostingClassifier(min_samples_leaf=20, n_estimators=200)  
gbc.fit(X_train, y_train)
```

▼ GradientBoostingClassifier
GradientBoostingClassifier(min_samples_leaf=20, n_estimators=200)

Feature importance

```
[14] import pandas as pd

pd.DataFrame(dict(Feature=gbc.feature_names_in_,
                  Value=gbc.feature_importances_))\
    .sort_values(by='Value', ascending=False)\
    .head(20)
```

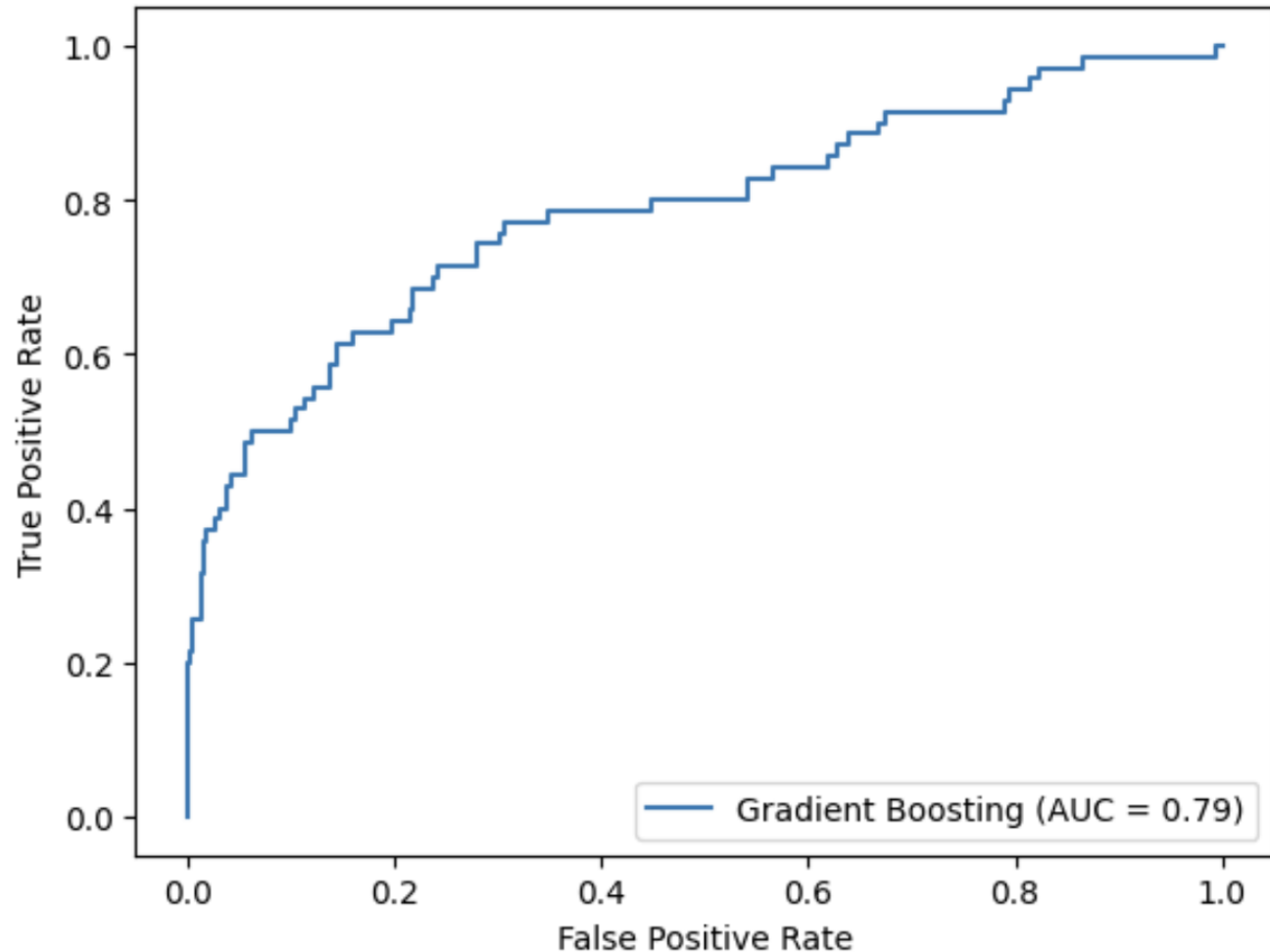


	Feature	Value
30	MonthlyIncome	0.128732
20	OverTime_Yes	0.103441
22	DailyRate	0.058398
21	Age	0.055890
25	EnvironmentSatisfaction	0.046479
40	YearsAtCompany	0.045058
36	StockOptionLevel	0.043650
23	DistanceFromHome	0.041710
32	NumCompaniesWorked	0.039302
31	MonthlyRate	0.038362

Evaluation

```
▶ y_res = gbc.predict_proba(X_test)
   fpr, tpr, thresholds = roc_curve(y_test, y_res[:,1])
   roc_auc = auc(fpr, tpr)
   display = RocCurveDisplay(fpr=fpr, tpr=tpr, roc_auc=roc_auc,
                             estimator_name='Gradient Boosting')
   display.plot()
```

↳ <sklearn.metrics._plot.roc_curve.RocCurveDisplay at 0x7f246e09a130>





eXtreme Gradient Boosting (xgboost)

xgboost is an optimized version of gradient boosting with these added features

- Regularization – xgboost provides both L1 and L2 regularization in model training
- Sparsity awareness – Incorporate missing data as criteria in feature selection
- Parallelization – allow parallelism of algorithm in either parallel or distributed environment
- Cache aware access – the learning algorithm efficiently utilizes the cache

Prepare parameters

-

```
[63] import xgboost as xgb

      param = {
          'eta': 0.3,
          'max_depth': 6,
          'objective': 'binary:logistic'
      }

      steps = 100
```

XGBoost Parameters

XGBoost Parameters

Before running XGBoost, we must set three types of parameters: general parameters, booster parameters and task parameters.

- **General parameters** relate to which booster we are using to do boosting, commonly tree or linear model
- **Booster parameters** depend on which booster you have chosen
- **Learning task parameters** decide on the learning scenario. For example, regression tasks may use different parameters with ranking tasks.
- **Command line parameters** relate to behavior of CLI version of XGBoost.

<https://xgboost.readthedocs.io/en/latest/parameter.html>



Prepare data

DMatrix is a internal data structure that used by XGBoost which is optimized for both memory efficiency and training speed.

Parameters

- **data** (*os.PathLike/string/numpy.array/scipy.sparse/pd.DataFrame/* – dt.Frame/cudf.DataFrame) – Data source of DMatrix. When data is string or os.PathLike type, it represents the path libsvm format txt file, csv file (by specifying uri parameter 'path_to_csv?format=csv'), or binary file that xgboost can read from.
- **label** (*list, numpy 1-D array or cudf.DataFrame, optional*) – Label of the training data.
- **missing** (*float, optional*) – Value in the input data which needs to be present as a missing value. If None, defaults to np.nan.
- **weight** (*list, numpy 1-D array or cudf.DataFrame, optional*) – Weight for each instance.

Note

For ranking task, weights are per-group.

In ranking task, one weight is assigned to each group (not each data point). This is because we only care about the relative ordering of data points within each group, so it doesn't make sense to assign weights to individual data points.

- **silent** (*boolean, optional*) – Whether print messages during construction
- **feature_names** (*list, optional*) – Set names for features.
- **feature_types** (*list, optional*) – Set types for features.
- **nthread** (*integer, optional*) – Number of threads to use for loading data from numpy array. If -1, uses maximum threads available on the system.

Modeling

```
[63] import xgboost as xgb

      param = {
          'eta': 0.3,
          'max_depth': 6,
          'objective': 'binary:logistic'
      }

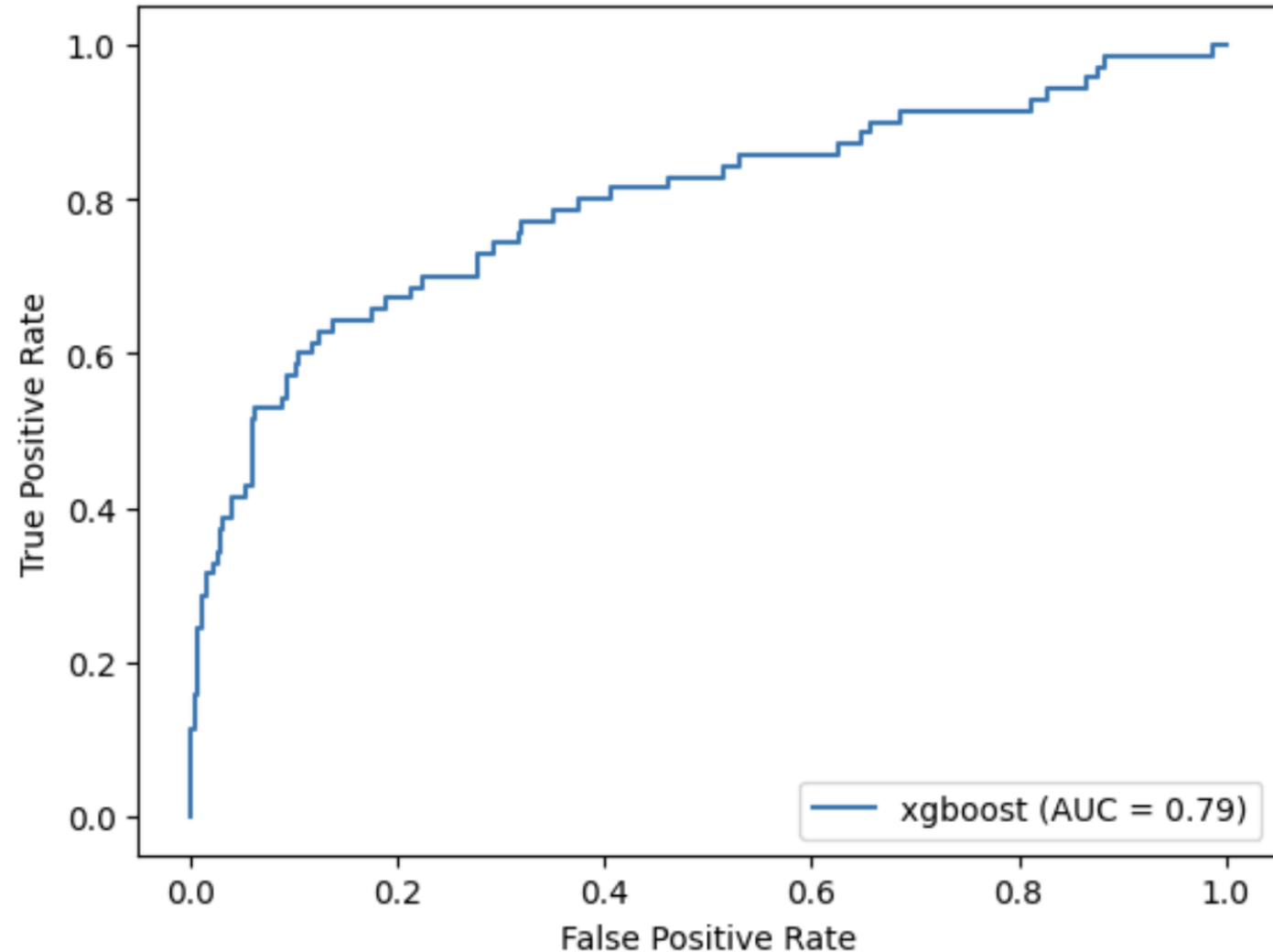
      steps = 100
```

Evaluation



```
fpr, tpr, thresholds = roc_curve(y_test, p)
roc_auc = auc(fpr, tpr)
display = RocCurveDisplay(fpr=fpr, tpr=tpr, roc_auc=roc_auc,
                          estimator_name='xgboost')
display.plot()
```

<sklearn.metrics._plot.roc_curve.RocCurveDisplay at 0x7f245a17c250>



Feature importance

```
▶ sorted(model.get_score(importance_type='total_gain').items(),  
         key = lambda x:x[1], reverse = True)
```

```
☞ [('MonthlyIncome', 167.65269470214844),  
   ('OverTime_Yes', 119.00450897216797),  
   ('DailyRate', 108.45510864257812),  
   ('MonthlyRate', 103.09760284423828),  
   ('Age', 94.56500244140625),  
   ('DistanceFromHome', 87.48432159423828),  
   ('HourlyRate', 86.21327209472656),  
   ('EnvironmentSatisfaction', 65.32698059082031),  
   ('NumCompaniesWorked', 58.19672775268555),  
   ('StockOptionLevel', 56.460105895996094),  
   ('YearsAtCompany', 49.44914245605469),  
   ('YearsSinceLastPromotion', 49.413124084472656),  
   ('TotalWorkingYears', 44.08100509643555),  
   ('PercentSalaryHike', 43.2405891418457),  
   ('RelationshipSatisfaction', 43.197994232177734),
```

Summary

model
generalized
มากขึ้น

- Ensemble learning allows us to generalize the machine learning model to further improve the model performance

ลด variance
ลด noise
เพิ่มประสิทธิภาพ

- Bagging is robust to noise

เรียนรู้จากข้อผิดพลาด
(ใช้ที่คนอื่นทำพลาด)

- Boosting allows us to squeeze the performance by modeling the residuals



End of Lecture 5

